Edurite College Shimoga

‖ Jnanam Uttamam ‖

Project Synopsis On :

COLLABRATIVE CODE EDITOR

SUBMITTED BY :

U06ED22S0040 MEET SONI
U06ED22S0042 DHANUSH D
U06ED22S0017 GANAVI D R
U06ED22S0004 VINAY N R

DATE :
28-02-2025

SUBMITTED TO :

Mrs.SAHITHYA D
Assistant Professor, BCA
Edurite College, Shimoga

HOD :

Mr. CHETAN N J
Dept of BCA
Edurite College, Shimoga

YEAR : 2024 - 25

INDEX

1. Introduction 3

2. Abstract 3

3. Features & Functionalities 4

4. Technology Stack 4

5. Literature survey 5

6. Proposed Methods 7

7. Models with Description 8

8. Flowchart 9

9. Expected outcome 9

10.Advantages 10

11. Conclusion 10

INTRODUCTION :

In the modern software development landscape, real-time collaboration is **essential for efficient teamwork and productivity. Traditional coding environments** often lack built-in communication tools, making it difficult for developers to work together seamlessly.

Our project, **"Real-Time Collaborative Code Editor",** is designed to bridge this gap by providing an interactive coding platform that enables multiple users to code simultaneously, communicate via voice chat, and use speech-to-text functionality for an enhanced development experience.

This **platform is user-friendly and efficient**, making it ideal for students, professionals, and remote teams. By integrating real-time synchronization, live voice communication, and a collaborative whiteboard, our project redefines remote pair programming and team-based coding, fostering a more engaging and intuitive software development process.

ABSTRACT :

The Real-Time Collaborative Code Editor is a **next-generation coding platform** that enables multiple developers to collaborate seamlessly in real time. It integrates live code synchronization, voice chat, speech-to-text functionality, and a collaborative whiteboard, ensuring efficient communication and teamwork.

Built with Electron.js, Node.js, Express.js, and WebSockets, the platform supports multi-user coding. Its secure access control and offline functionality make it suitable for students, professionals, and remote development teams.

By bridging the gap between coding and communication, this project enhances productivity, fosters an interactive learning environment, and redefines modern collaborative software development.

FEATURES AND FUNCTIONALITIES :

- **Real-Time Code Synchronization –** Uses WebSockets (Socket.IO) to enable seamless live coding between multiple users.
- **Voice Chat –** Integrated WebRTC-based voice communication for easy collaboration and discussions.
- **File Management –** Users can save, load, and manage files with local storage support.
- **Syntax Highlighting –** Powered by CodeMirror or Monaco Editor to improve readability and coding efficiency.
- **Compilation Support –** Allows execution of Java and Python code within the application.
- **Customizable Themes –** Multiple UI themes for enhanced personalization and accessibility.
- **Speech-to-Text –** Uses the Web Speech API to convert spoken words into code, improving accessibility and speed.
- **Collaborative Whiteboard –** A shared drawing space for brainstorming, flowcharts, and visual explanations in real time.

TECH STACK :

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js, Express.js
- **Database:** MongoDB
- **Real-Time Communication:** WebSockets (Socket.IO)
- **Desktop Application Framework:** Electron.js
- **Voice Chat & Speech-to-Text:** WebRTC & Web Speech API
- **Code Editor:** CodeMirror or Monaco Editor
- **Whiteboard:** Fabric.js or Konva.js for real-time collaborative drawing

LITERATURE SURVEY :

With the increasing demand for remote collaboration, real-time collaborative code editors have become essential tools for developers, educators, and teams working on software projects. These platforms enable multiple users to edit and execute code in real time, enhancing productivity and communication. This literature survey explores existing solutions, their features, limitations, and the technological advancements that contribute to the development of such systems.

EXISTING SOLUTIONS

Several collaborative code editors exist, each with unique functionalities. Below are some widely used platforms:

Google Colab

- Provides real-time collaboration on Jupyter notebooks.
- Supports Python execution with built-in GPU acceleration.
- Limited to Python and lacks broad multi-language support.

Visual Studio Code (Live Share)

- Enables real-time collaboration through Live Share extension.
- Supports multiple programming languages.
- Requires installation and is not browser-based.

Replit

- A browser-based collaborative coding platform.
- Supports multiple programming languages and real-time collaboration.
- Limited customization and lacks integrated voice chat features.

CodeSandbox

- Focuses on frontend development and allows real-time editing.
- Suitable for JavaScript-based projects.
- Lacks built-in voice chat and speech-to-text features.

LIMITATIONS OF EXISTING SYSTEMS:

While existing collaborative editors offer various features, they have limitations that our proposed solution aims to address:

- **Lack of Integrated Voice Chat:** Most platforms require third-party communication tools.
- **Limited Speech-to-Text Support:** Few editors support hands-free coding.
- **Inconsistent Multi-Language Compilation:** Many editors do not support direct execution of multiple programming languages within the same session.
- **Access Control & Security Concerns:** Some platforms lack robust user authentication mechanisms.

Proposed Solution

Our project addresses the limitations by:

- Implementing **real-time collaborative editing** using WebSockets.
- Integrating **WebRTC-based voice chat** for seamless communication.
- Adding **speech-to-text functionality** to enable voice-based coding.
- Supporting **multi-language execution** for Programming Languages.
- Providing **secure access control and authentication** mechanisms.

Conclusion :

The development of a real-time collaborative code editor with voice chat and speech-to-text capabilities enhances remote teamwork and accessibility. By leveraging technologies such as WebSockets, WebRTC, Electron.js, and the Web Speech API, this project aims to provide an efficient, user-friendly, and feature-rich collaborative coding experience.

References :

- **Google Colab:** https://colab.research.google.com/
- **Visual Studio Code Live Share**: https://visualstudio.microsoft.com/
- **Replit**: https://replit.com/
- **CodeSandbox**: https://codesandbox.io/
- **WebRTC Overview**: https://webrtc.org/
- **Electron.js**: https://www.electronjs.org/
- WebSockets Documentation: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- **Web Speech API**: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

PROPOSED METHOD

The proposed system is a collaborative code editor designed to facilitate real-time coding, voice communication, and whiteboard collaboration. The development process follows a structured approach to ensure seamless integration of all features.

1. Frontend Development

- The user interface (UI) will be built using HTML, CSS, and JavaScript, ensuring a responsive and user-friendly experience.
- CodeMirror or Monaco Editor will be integrated for syntax highlighting and an improved coding experience.
- UI customization will include multiple themes for accessibility.

2. Backend Implementation

- The backend will be developed using Node.js and Express.js to handle real-time collaboration and data processing.
- Session management will be implemented to store and retrieve ongoing collaborations.
- A MongoDB database will be used to store session details, user preferences, and access control settings.

3. Real-Time Synchronization

- WebSockets (Socket.IO) will be used to synchronize code edits across multiple users instantly.
- A collaboration link will be generated for users to invite others to join the session.
- Multi-user cursors and presence indicators will be implemented to show who is editing.

4. Desktop Application Support

- The platform will be packaged as a desktop application using Electron.js to support offline functionality.
- Local file storage and management will allow users to save and load files easily.

5. Voice & Speech Integration

- WebRTC will be used to enable real-time voice chat, reducingdependency on external communication apps.
- The Web Speech API will allow speech-to-text conversion, enabling users to code using voice input.

6. Collaborative Whiteboard

- A real-time shared whiteboard (using Fabric.js/Konva.js) will be integrated to support diagrams, flowcharts, and brainstorming.

7. Security & Access Control

- Secure authentication will be implemented to prevent unauthorized access.

This proposed method ensures efficient, real-time collaboration while maintaining security, performance, and accessibility.

Models with Description

To better understand the functionalities and architecture of a real-time collaborative code editor, the following models are considered:

Client-Server Model

- A centralized server manages all client requests and synchronizes code changes.
- Ensures consistency and security but may introduce latency.

Peer-to-Peer (P2P) Model

- Direct communication between users without a central server.
- Reduces latency but may face synchronization challenges.

Operational Transformation (OT) Model

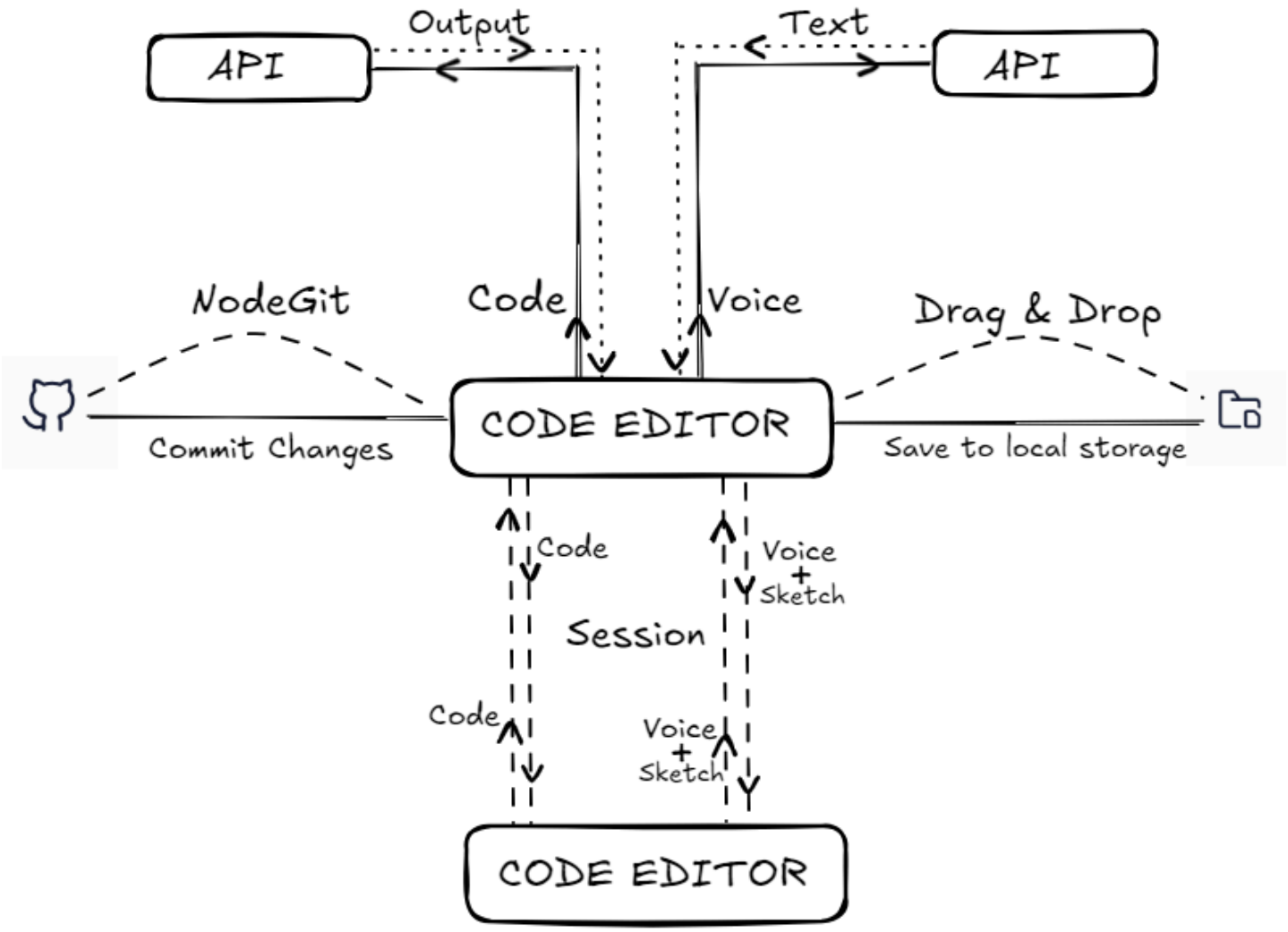
- Algorithm to handle concurrent editing and merge changes efficiently.
- Used in Google Docs and similar real-time collaborative tools.

Conflict-Free Replicated Data Type (CRDT) Model

- Ensures consistency in distributed environments without requiring a central server.
- Reduces dependency on real-time server availability.



Flowchart -



EXPECTED OUTCOMES

- A seamless collaborative experience with live code synchronization with whiteboard.
- Secure access control and authentication for users.
- Enhanced communication through integrated voice chat.
- Speech-to-text capabilities to assist in coding without manual typing.
- Improved user engagement through UI themes and syntax highlighting.
- A scalable and extensible system that can be expanded with additional programming languages and features in the future.

ADVANTAGES :

Real-Time Collaboration – Enables multiple users to code together seamlessly.
Integrated Voice Chat – Enhances communication without relying on external apps**.**
Real-Time Collaboration – Enables multiple users to code together seamlessly.
Integrated Voice Chat – Enhances communication without relying on external apps.
Speech-to-Text Input – Makes coding more accessible and faster.
Multi-Language Compilation – Allows execution of Java and Python code within the app.
Secure Access Control – Ensures only authorized users can join sessions.
Customizable UI Themes – Improves user experience with multiple theme options.
Syntax Highlighting – Enhances readability and debugging.
Efficient File Management – Supports saving and loading of files locally.
Cross-Platform Support – Works on Windows, macOS, and Linux via Electron.js.
Ideal for Education & Teams – Useful for students, educators, and professional developers.

CONCLUSION :

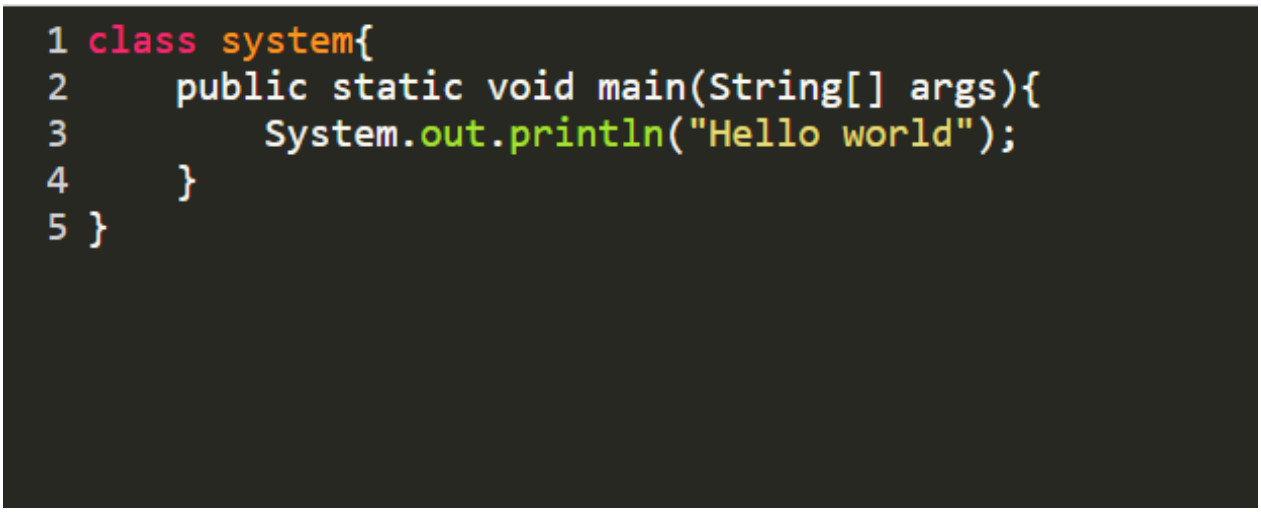
The **"Real-Time Collaborative Code Editor"** is a significant step toward enhancing the way developers work together. By integrating real-time collaboration, voice chat, and speech-to-text functionalities, our project offers an **innovative and practical solution** for coding teams, students, and professional developers. This project not only enhances productivity but also fosters an intuitive learning environment for aspiring programmers. With the advancement of collaborative technologies, our platform sets a new standard in remote and team-based software development.

1. EDITOR WITH SYNTAX HIGHLIGHTING :

A code editor is a tool designed for writing and managing code efficiently. This editor is built using CodeMirror, a powerful JavaScript library that provides essential coding features, making the development process smoother and more efficient.

Key Features

- **Syntax Highlighting** – Colors different code elements for better readability.
- **Line numbers** - auto increment of line numbers on creating new line
- **Auto Bracket Closing** – Automatically inserts closing brackets to reduce syntax errors.
- **Automatic Indentation** – Ensures proper code formatting for better structure.
- **Breakpoints** - Helps pointing at a particular line



2. SAVE CODE FUNCTIONALITY

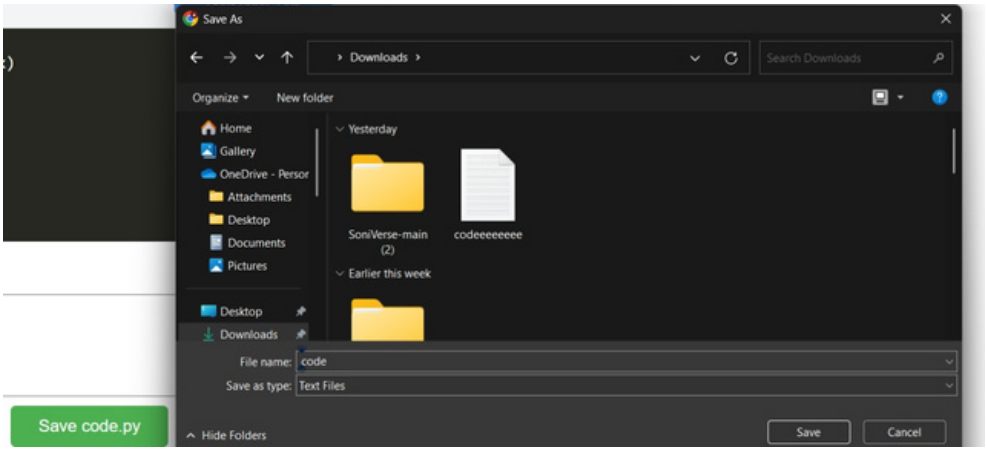
The Save Code button in our code editor enables users to store their code locally, ensuring their work is preserved for future use. This feature allows file management, preventing data loss and enhancing productivity.

How It Works :

When users click the Save Code button, the system opens the file manager, prompting them to choose a directory and file name. Once confirmed, the code is saved in the selected format and location.

Key Features :

- **File Manager Integration**: Users can select where to save files.
- **Multiple Format Support**: Supports formats like .txt, .js, .py, .cpp, etc.
- **Overwrite Warning**: Prompts users before overwriting an existing file.
- **Data Safety:** Prevents accidental data loss.
- **User Control:** Allows custom file names and locations.
- **Convenience**: Easy and quick file storage.
- **Data Protection**: Prevents loss due to accidental closure.
- **Customization**: Users control file organization.
- **Improved Workflow**: Enhances coding efficiency.

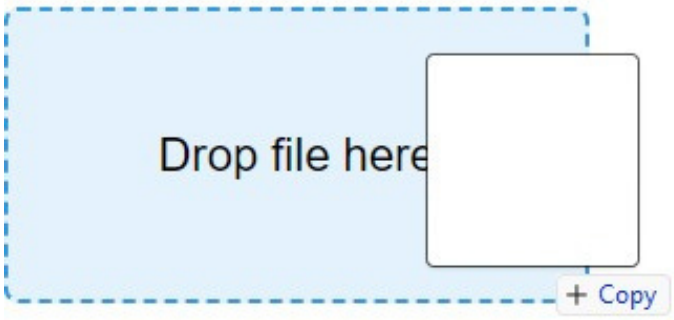


Step-by-Step Process :

- **User Writes Code:** Code is typed in the editor.
- **Click Save Button:** The save function is triggered.
- **Open File Manager:** Users choose where to store the file.
- **Select File Name & Format:** A name and format are set.
- **Confirm & Save:** The file is stored successfully.
- **Confirmation Message:** An alert verifies successful saving.

3.Drag and Drop File

The drag-and-drop file feature in a compiler simplifies the process of uploading and compiling code files. This functionality allows users to conveniently drag files from their local system and drop them into the compiler interface, eliminating the need for manual file selection through traditional input dialogs.



Features :

- **User-Friendly –** Drag-and-drop for easy file uploads.
- **Multi-Language Support –** Works with C, C++, Java, Python, and JavaScript.
- **Auto File Detection –** Identifies file types and applies correct rules.
- **Error Handling –** Instant feedback on invalid files or failures.
- **Real-Time Compilation –** Runs code immediately after upload.
- **Cross-Browser –** Works smoothly on all modern browsers.
- **Secure –** Blocks malicious files and prevents unauthorized access.

Advantages :

- **Efficiency** – Saves time by eliminating manual file browsing.
- **Productivity** – Streamlines workflow for handling multiple files.
- **Error Reduction** – Reduces mistakes in file selection.
- **User Experience** – Provides a smooth and modern file upload process.

4.Code Compilation

This is a server-side API endpoint (/compile) that enables online code compilation and execution for multiple programming languages. It integrates with the Piston API to run user-submitted code and return the output. The endpoint supports Python and Java and can handle user inputs.

Features :

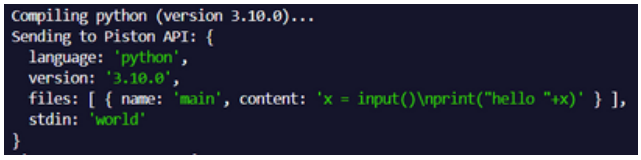
- Accepts source code, language, and input dynamically via API requests.
- Uses Piston API for execution and retrieves the output.
- Supports multiple programming languages with version control.
- Provides error handling and detailed logging for debugging.

Advantages :

- Execution time is also been calculated between API request
- Supports multiple languages with different versions.
- Handles user input and returns execution results instantly.
- Offers detailed error logs for better debugging.

This API simplifies cloud-based code execution, making it useful for online compilers, coding platforms, and learning environments. It enhances flexibility by supporting multiple languages and ensures robustness with detailed error handling.







5. Collabration across devices

This collaborative code editor uses Socket.IO for real-time synchronization and Ngrok to expose the server for external access. It allows multiple users to edit code simultaneously, ensuring seamless collaboration.

Features :

- **Real-time Code Editing**: Syncs code changes across all connected users.
- **Remote Accessibility:** Uses Ngrok to make the server publicly accessible.
- **Instant Updates:** Changes reflect immediately using WebSockets.
- **Multi-User Collaboration:** Supports multiple users in the same session.
- **Low Latency:** Real-time updates with minimal delay.
- **Easy Access**: Ngrok removes the need for complex server configurations.
- **Efficient Communication**: WebSockets provide smooth, event-driven collaboration.
- **Scalability**: Can be extended with additional features like voice chat and version control.

How It Works :

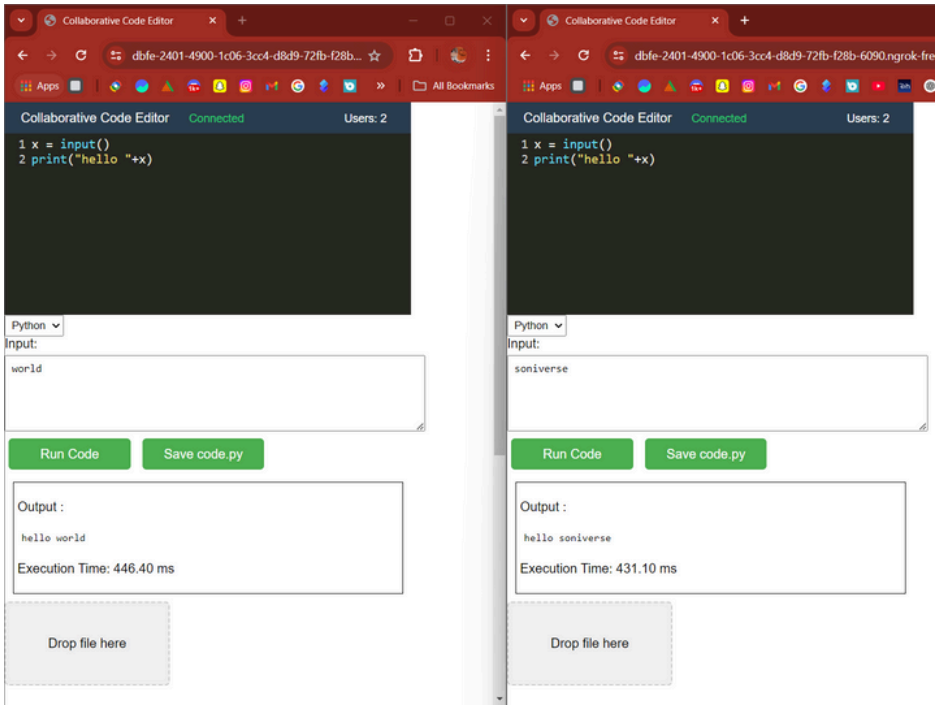
1. Server Setup - A Node.js server is created using Express.js. Socket.IO is initialized to handle real-time communication.

2. Ngrok Integration - Ngrok is installed and started to expose the local server. It generates a public URL that clients can use to connect remotely.

3. Client-Side Connection -The frontend (HTML, CSS, JavaScript) connects to the Socket.IO server.A unique room ID is used to group users in the same session.

4. Real-Time Code Synchronization - When a user types in the editor, an event is sent to the server.The server broadcasts this update to all users in the same room.Other users receive the update and apply it to their editors.

5. Multi-User Collaboration - Users can join or leave rooms dynamically. Changes made by any user in the room are reflected for all participants.



This setup provides a fast, accessible, and efficient way to build a real-time collaborative coding environment. With Socket.IO handling synchronization and Ngrok ensuring external access, it is ideal for online coding platforms and team projects

6. Coding Sessions

The Collaborative Code Editor enables multiple users to work together in real time on the same code. This feature allows developers to generate a unique session code and share it with others for seamless collaboration. It enhances teamwork and efficiency in coding projects.

Key Features:

1. Session Generation & Joining

- Users can generate a new session with a unique verification code.
- Others can join by entering the shared code

2. Real-Time Code Collaboration

- Multiple users can write, edit, and review code simultaneously.
- Changes are instantly reflected across all connected users.

3. Secure and Unique Session Code

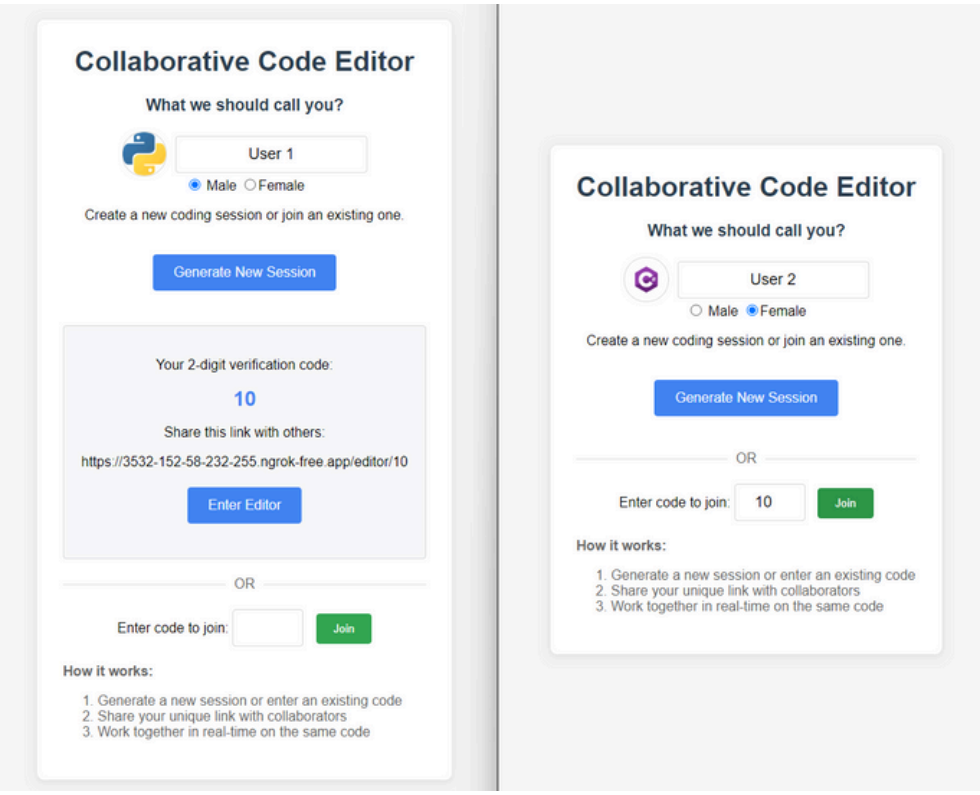
- Each session has a unique code and code for private collaboration.
- Users can share the session securely with team members.

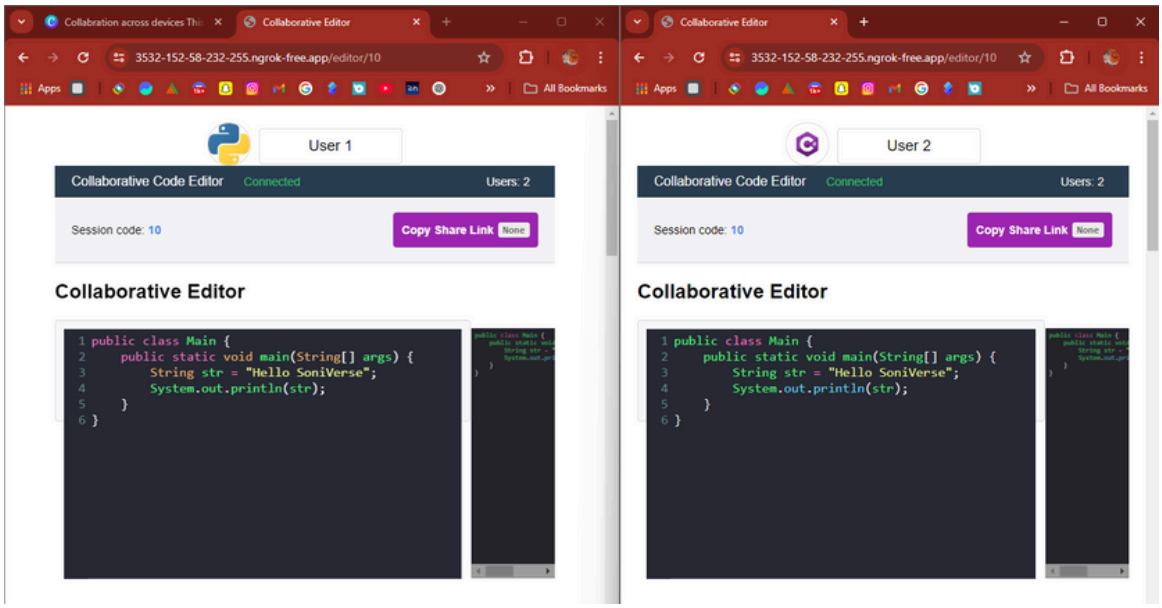
4. User-Friendly Interface

- Simple and easy-to-use interface for both beginners and professionals.
- Options to create, join, and collaborate with minimal effort

5. User Count

- Each time a user joins or leaves, the count updates dynamically.





How It Works

- Generate a new session or enter an existing session code.
- Share your unique code with collaborators.
- Work together in real time on the same code file.

The Collaborative Code Editor is a powerful tool for team-based programming, allowing real-time sharing and editing of code. It enhances productivity, teamwork, and efficient coding collaboration for developers, students, and professionals.

7. Voice Chat Feature

The Voice Chat Feature in the Collaborative Code Editor enables users to communicate in real-time while coding together. This feature enhances teamwork by allowing developers to discuss ideas, troubleshoot issues, and coordinate coding efforts without needing external applications. The built-in voice communication ensures seamless collaboration, improving productivity and efficiency.

Key Features

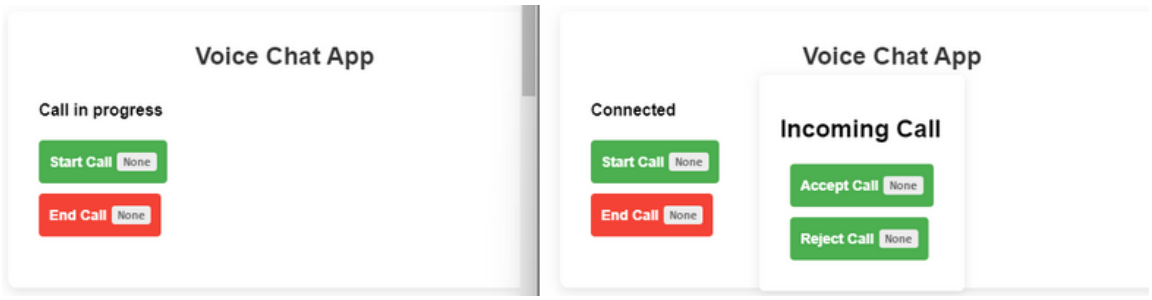
1. Real-Time Voice Communication
- Users can start a voice call with connected collaborators.
- Enables direct discussions while coding for better coordination.

2. Call Request & Incoming Call Popup
- When a user initiates a call, the recipient receives an Incoming Call popup.
- The popup provides options to Accept or Reject the call.

3. Session-Based Voice Chat
- Voice chat is linked to active coding sessions.
- Only users in the same session can communicate via voice.

4. Start and End Call Options
- Users can start a voice chat by clicking the Start Call button.
- Calls can be ended anytime using the End Call button.

5. Improved Collaboration & Productivity
- Eliminates the need for external communication tools like Zoom or Discord.
- Helps in faster debugging, brainstorming, and teamwork.



How It Works

- A user clicks Start Call to initiate a voice chat.
- The recipient gets an Incoming Call popup with Accept and Reject options.
- If accepted, both users can communicate in real-time.
- Users can end the call anytime by clicking End Call.

The Voice Chat Feature is a valuable addition to the Collaborative Code Editor, enhancing real-time communication, teamwork, and coding efficiency. It ensures smooth interactions, making the coding process more interactive and productive.

8. Chat Box Feature

The Join Chat Feature in the Collaborative Code Editor allows users to communicate through text messages while working on code. This feature enables seamless interaction between collaborators, making it easier to discuss ideas, share updates, and troubleshoot issues in real-time. It eliminates the need for external chat applications, providing a built-in messaging system within the coding environment. The Media Sharing and Download Feature in the chat system enhances communication by allowing users to send and receive voice notes, PDFs, photos, and videos. Additionally, users can download shared media for future reference.

Key Features

1. Username-Based Chat Access

- Users can enter a unique username to identify themselves in the chat.
- Ensures personalized communication between team members.

2. Instant Messaging

- Allows users to send and receive messages in real-time.
- Facilitates smooth discussions without interrupting the coding workflow.

3. Seamless Integration with Coding Sessions

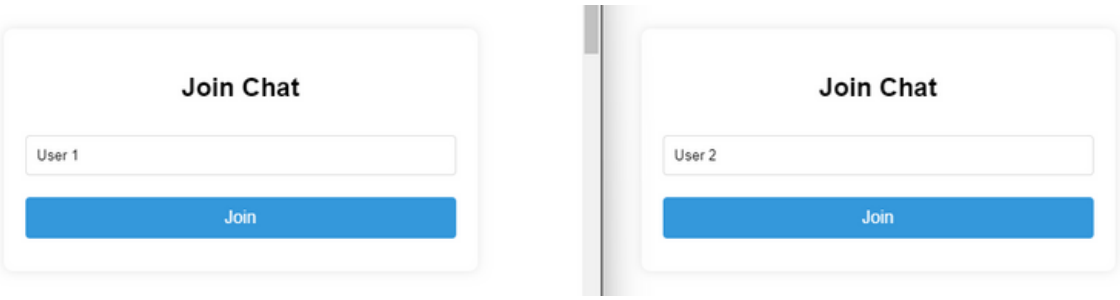
- Chat is linked to the coding session, so only session participants can communicate.
- Helps maintain focused and relevant discussions.

4. User-Friendly Interface

- Simple "Join Chat" button for quick access.

5. Real-Time Collaboration & Productivity Boost

- Enables teams to share code-related queries, provide feedback, and resolve errors quickly.



9. Media Sharing and Download Feature in Chat

The Media Sharing and Download Feature in the chat system enhances communication by allowing users to send and receive voice notes, PDFs, photos, and videos. Additionally, users can download shared media for future reference.

Key Features

1. Sending Voice Notes

- Users can record and send voice messages directly from the chat.
- Helps in quick communication without typing.
- Ideal for explaining complex topics more effectively.

2.Sharing PDFs

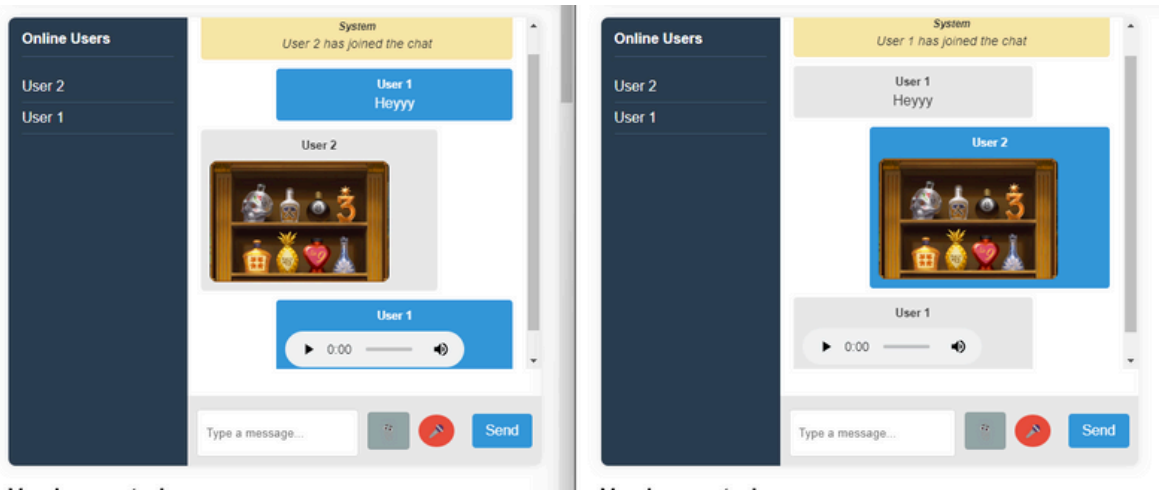
- Users can send PDF documents to share notes, project files, or reports.
- Ensures easy document exchange within the chat.

3. Sending Photos and Videos

- Allows users to share images and videos for better visualization.
- Helps in collaborative work and project discussions.

4. Downloading Shared Media

- Users can download voice notes, PDFs, photos, and videos.
- Ensures offline access to important files when needed.



How It Works

- Users select a file (voice note, PDF, image, or video).
- The selected media is sent to the chat in real time.
- Other users can view and download the shared content instantly.

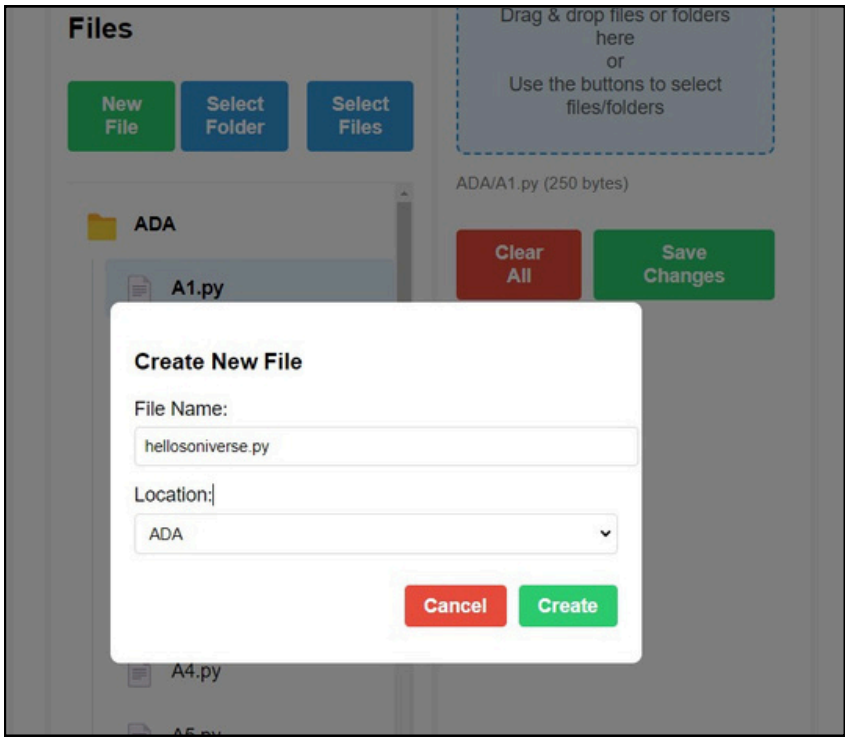
The Media Sharing and Download Feature enhances user interaction by integrating multiple media types for seamless communication. It ensures better collaboration, efficient file exchange, and easy media accessibility in the chat system.

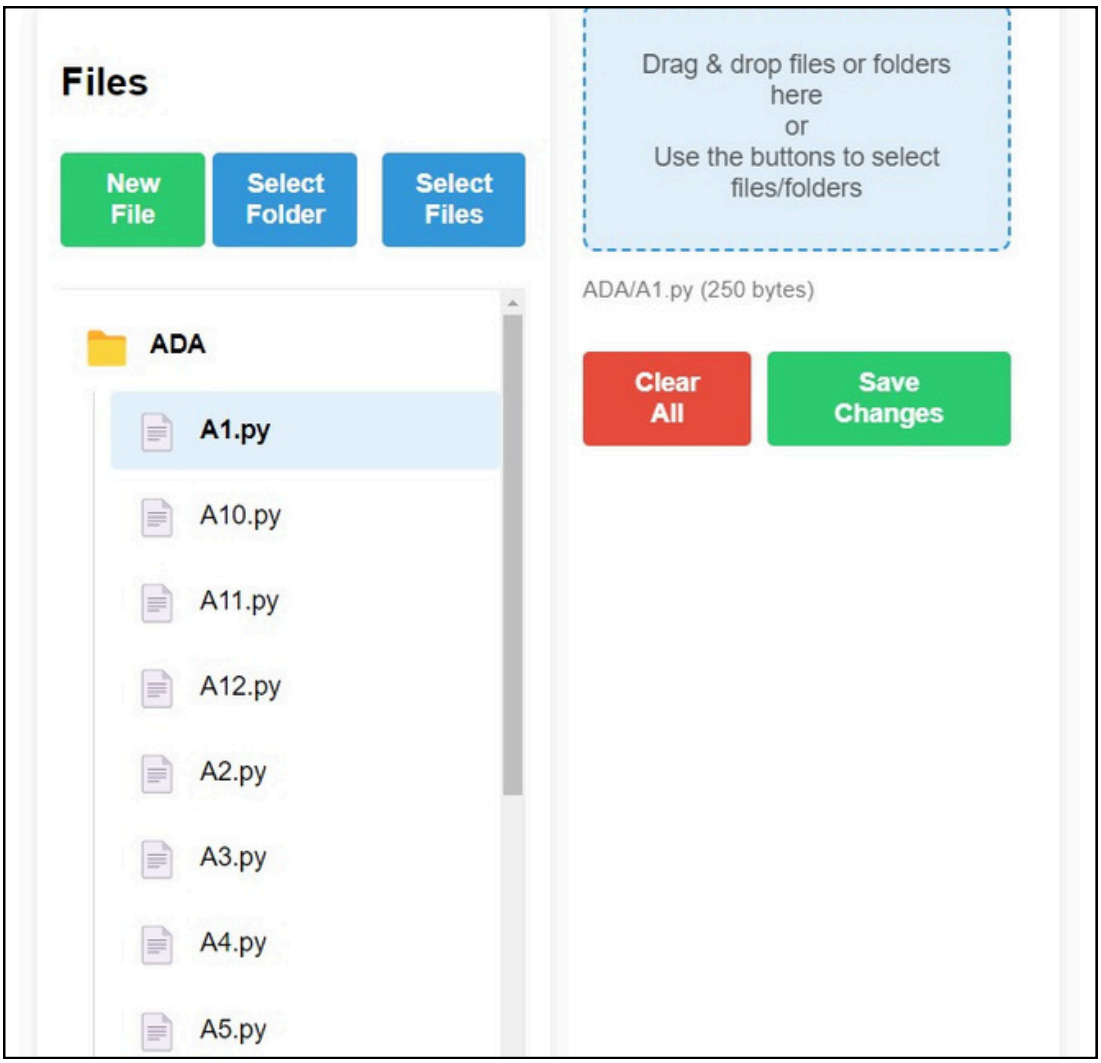
10. File Creation Feature

This module is a core part of the code editor interface, focusing on efficient file management and code-saving functionality. The feature allows users to create, organize, and manage all source code files like python, dart, and 20 more within a selected folder structure, enhancing user experience especially during coding and managing multiple packages.

Key Functionalities:

1. **Select Folder -** “Select Folder” button helps to access local storage of PC and select wished folder, the respected files of folder will be displayed on left side of panel.
2. **Selection of single/mutiple file** - Is also allowed if the whole folder is not required, we can still access it.
3. **File creation** - “New File” button will help us create a new file in the selected folder. A popup will be displayed on screen, once we put file name with its extension, create button will generate a empty file in desired location(folder). cancel button will abort the activity of creating new file.
4. **Code saving options** - If we had an edit in file, save button will re-write the content to same file sand save it permanently.
5. **Clear all** - Button will clear the directory structure, it’ll remove the folder and files that we choosed.
6. **File Size -** When we choose files, it’s size in bytes/KB/MB will be displayed.





How it Works

1. Drag and Drop Handling:

- When a folder or file is dropped, the handleDrop() function intercepts it.
- It checks if the dropped item is a folder using webkitGetAsEntry() and recursively parses the folder structure using traverseFileTree().

2. Building the Folder Tree:

- A nested folderStructure object is built from the parsed directories and files.
- Each file's path, type, and content are stored in the loadedFiles object using file.text().

3. Rendering Files and Folders:

- The folder structure is rendered visually using recursive HTML creation.
- Each file shows an icon based on its type, and clicking a file loads it into the appropriate editor.

4. Editor Integration:

- The app uses three dedicated code editors (html-editor, css-editor, js-editor).
- When a user clicks on a file (e.g., .html, .css, .js), the file content is displayed in the correct editor, and the active file is tracked.

5. Saving Files:

- Users can save changes either using the File System Access API (if supported) or fallback to downloading modified files.
- The saveFile() function writes content back to disk or triggers download using a Blob.

Benefits:

- **User-Friendly Interface :** Simplifies the file handling process.
- **Efficient Workflow :** Suitable for real-time coding, testing, and saving.
- **Perfect for Collaboration :** When combined with features like Socket.io, this file structure supports team development and editing.

Ideal Use Cases

Coding assignments, collabration with friends.

11. Version Control Feature

This feature provides built-in Git version control functionality in the code editor interface, allowing users to manage source code changes directly from the platform. It eliminates the need to switch to an external Git client and simplifies the process of pushing code to platforms like GitHub.

Key Functionalities:

1. Personal Access Token Integration:

- Users input a GitHub personal access token to authenticate securely.
- Ensures privacy and control over repositories.

2. Repository Configuration:

- Repository Owner (e.g., technophileee)
- Repository Name (e.g., Tester)
- File Path (e.g., readme.md)
- Enables targeted file commits to a specific repository.

3. Commit Message Input:

- Users can write a custom commit code changes.

4. One-Click Commit:

- The “Commit Changes” button allows easy pushing of changes.

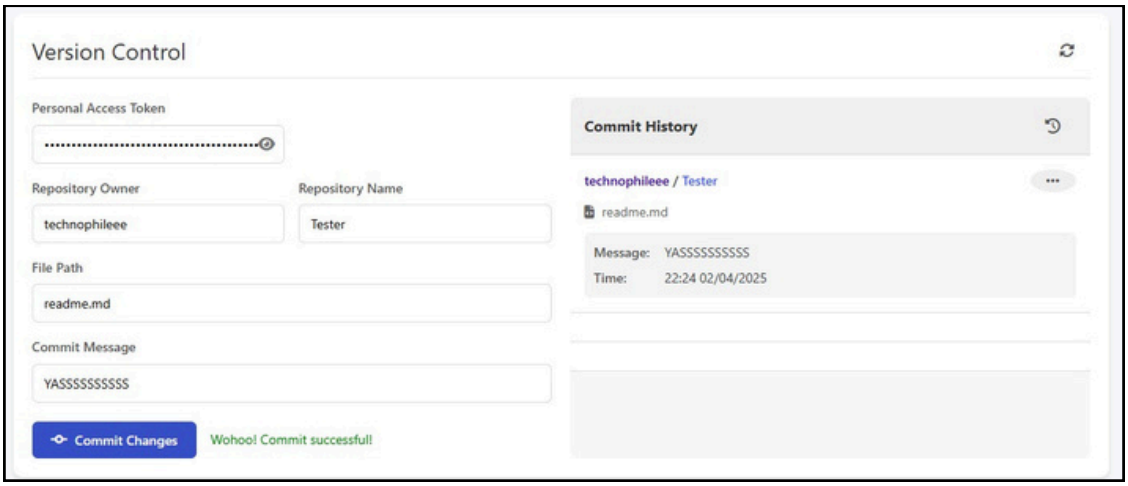
5. Shows a success message:

- if code updated in github repository "Woohoo! Commit successful!" message will be displayed.

6. Commit History Display:

Displays recent commits with:

- Message
- Time
- File committed
- Helps track and review changes visually.



How it works

1. Auto-Commit Process

When the form is submitted, the frontend collects GitHub credentials and file content, then initiates a commit request to the backend /commit route.
Here's how the backend handles the process:

- Step-by-step GitHub Commit Flow:
  - Token Authentication:
  - The backend uses the decrypted personalAccessToken to authenticate with the GitHub API.
  - SHA Retrieval:
  - It fetches the file's current metadata from the GitHub repository (via a GET request) to obtain the latest sha. This is required to overwrite or update the file.
- A payload is constructed with:
  - Message: The commit message from the form.
  - Content: The file content, encoded in Base64.
  - SHA 256 algorithm: To ensure GitHub knows which file version is being updated.
  - Branch: Hardcoded as main for now.

- Final Commit Request:
- A PUT request is made to GitHub's content API to commit the new file content to the specified repo path.
- Response Handling:
- If successful, the backend returns a success message with commit metadata. Otherwise, it logs the error and returns a meaningful error message to the frontend.

2**. Secure Local Storage of Personal Access Token:**

- The encrypted token, along with GitHub details (owner, repo, file path, message, time), is stored in localStorage so users don't have to enter credentials repeatedly.
- When the user submits their GitHub Personal Access Token, it's first encrypted using a custom algorithm that:
- Cycles through predefined shift values (3, 5, 7) to slightly alter character codes.
- Applies an XOR operation with a rotating set of keys (111, 123, 145) for obfuscation.
- Encodes the final encrypted string into Base64 for safe storage.
- During token reuse (e.g., form refill), the encrypted token is:
- Decoded from Base64.
- XOR reversed and shifted back using the same keys to get the original token.

3**. Commit History Display**

- On clicking Get History, the stored data (from localStorage) is displayed, helping users keep track of their recent commits.

4**. One-Click Form Refill**

- Clicking Fill VC auto-fills the form with previously used and securely stored data, improving user experience.

5**. Reset Button**

The Reset VC button clears all form inputs, allowing a fresh start.

Benefits:

- Simplifies Git Workflow: No command line required.
- Real-Time Feedback: Success confirmation and logs.
- Good for Team Projects: Helps maintain collaboration and transparency.
- Encourages Best Practices: Promotes the habit of committing with proper messages.
- Shared Academic assignments with all.

12. Customize Key Bindings Feature

The Customize Key Bindings feature is an essential part of improving user productivity and workflow in coding environments. This functionality allows users to assign their own keyboard shortcuts for common actions like compiling code, running programs, saving files, committing changes, and more.

This feature enhances the user experience by allowing personalization. Instead of relying on default shortcuts, users can define combinations that are easier or more comfortable for them. For example, shortcuts can be set for actions such as: Compile: **Ctrl + `,** Run: **Ctrl + R**

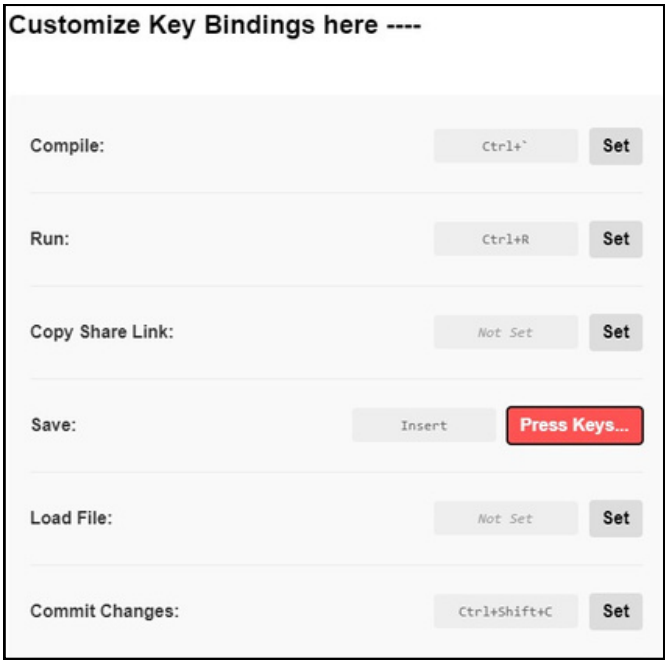
The system provides a simple interface with a “Set” button beside each command. Users can click **"Set"** and press the desired key combination, which will be saved and used for that action moving forward. If a shortcut is already set, it is displayed next to the action.

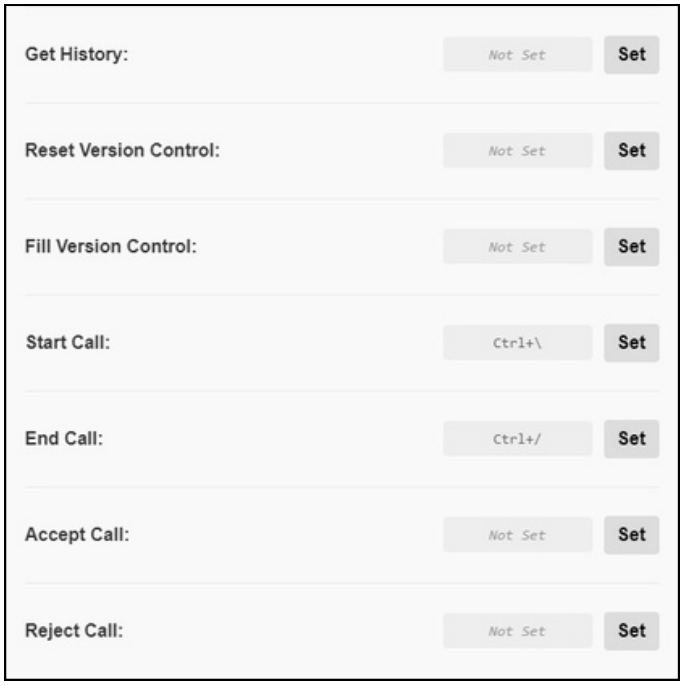
Key Features:

- Set custom shortcuts for any command.
- Improve coding speed with personalized key binds.
- Easy-to-use interface for assigning keys.
- Supports development, version control, and call operations.
- Makes the environment user-friendly and efficient.

How It Works:

- **Actions Defined:** Each action (e.g., Compile, Save) has an ID, name, button, and shortcut display element.
- **Keybindings Storage:** A keybindings object stores user-defined shortcuts for each action.
- **UI Generation:** GenerateKeybindingsUI() creates rows with "Set" buttons to assign shortcuts.
- **Key Recording:** When "Set" is clicked, it waits for a key combo and saves it (e.g., Ctrl+S).
- **Duplicate Check:** Prevents assigning the same combo to multiple actions.
- **Execution:** On key press, handleKeyDown() checks if any combo matches and calls executeAction().
- **Visual Feedback:** Logs actions, shows shortcuts, and briefly animates buttons.





Benefits -

- **Faster Workflow:** Users perform actions quickly without leaving the keyboard.
- **Personalized Control :** Adapts to user habits and preferences.
- **Accessibility :** Helps users with disabilities or unique setups.
- **Consistency Across Tools:** Users can match keybinds with other editors/tools they use.
- **Reduces Repetition :** Saves time on frequently used actions like saving, formatting, or switching themes.

13. Collaborative Whiteboard

The Collaborative Whiteboard is a powerful real-time drawing tool designed for interactive learning and live coding explanations. It enables users to visually represent data structures, flowcharts, or logic diagrams while collaborating with others over the network using WebSockets.

As seen in the images, the whiteboard allows users to draw, write, and illustrate concepts using a variety of tools:

Key Features:

1. Brush Tool:

- Multiple color options (black, red, blue, green) with customizable brush size for drawing diagrams or highlighting points.

2. Eraser Tool:

- Adjustable eraser to remove specific parts of the drawing.

3. Text Tool:

- Users can input text of different sizes and colors, helpful for labeling elements in diagrams.

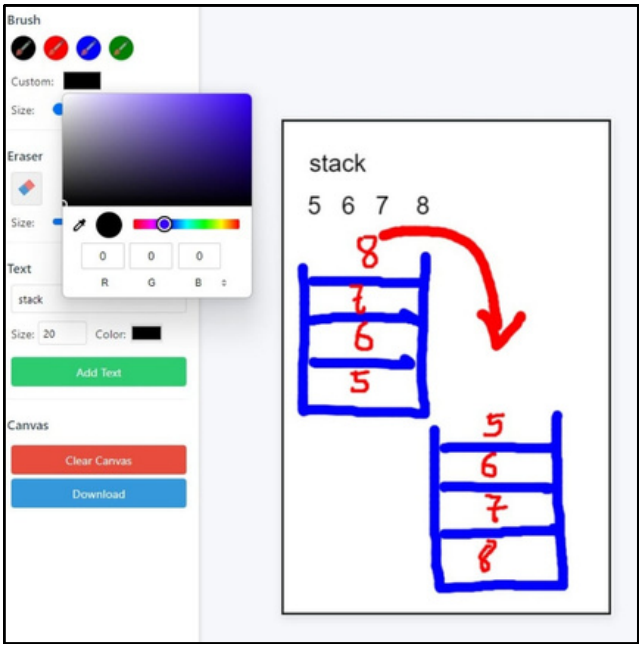
4. Canvas Controls:

- Clear Canvas to reset the board instantly.
- Download to save the whiteboard as an image for offline reference or reports.
- The core functionality is powered by Socket.io, allowing all connected users to see the updates instantly.

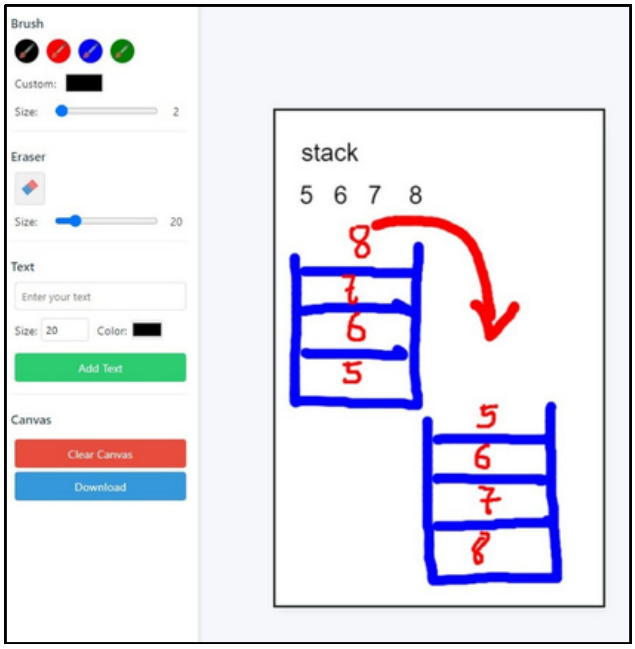
Benefits :

- Pair programming
- Remote team discussions
- Teaching data structures
- Explaining code logic live

User 1



User 2



How It Works:

- **Canvas Setup:** Initializes canvas with mouse drawing, custom color, and brush size controls.
- **Drawing Logic:** On mouse move while pressed, lines are drawn between points and emitted via Socket.IO.
- **Real-time Sync:** Other users receive drawing data and render the same lines to stay in sync.
- **Color & Eraser:** 5–6 predefined colors, eraser toggle with custom size and cursor icon.
- **Clear Canvas:** Button clears board and notifies all users via socket.
- **Save Feature:** Users can save canvas as PNG directly to downloads.
- **Persistence:** Canvas auto-saves to localStorage before reload and restores on load.
- **Text Tool:** Users add text by clicking canvas after entering input–syncs live to all peers.

In the images provided, a stack operation (push/pop) is visually explained using the whiteboard. This is especially useful for students or peers who learn better through visual aids rather than just code.
The collaborative nature ensures everyone stays on the same page, making it more than just a drawing board–it becomes a shared live space for visual coding communication.

14. Added 12 more Programming languages

The Multi-Language Code Editor is a powerful, browser-based platform that enables users to write, test, and visualize code in various programming languages. It features intuitive design, real-time syntax highlighting, and version tracking, making it ideal for both learners and developers.

Key Features:

1. Multi-Language Support (Total 14 Programming Languages):

- The editor allows users to switch between popular languages with one click, each accompanied by its current version:
- C v10.2.0
- C++ v10.20.0
- Java v15.0.2
- Dart 2.19.6
- Python 3.10.0
- JavaScript 5.0.3
- TypeScript v5.0.3
- Go v1.16.2
- Rust v1.16.8
- R v4.1.1
- PHP v8.2.3
- Ruby v3.0.1
- Swift 5.3.3
- Kotlin 1.8.20

2. Real-Time Syntax Highlighting:

- Automatically adjusts highlighting based on selected language, improving code readability and debugging experience.

3. We have added 12 more languages:

- Using same approach, Piston API is used to transfer input and code, API response will be in JSON format which will be extracted and displayed on output box.

4. Sample Code:

- Seperate buttons of Langaguges will be converted into dropdown and each option emits sample code on editor



Benefits:

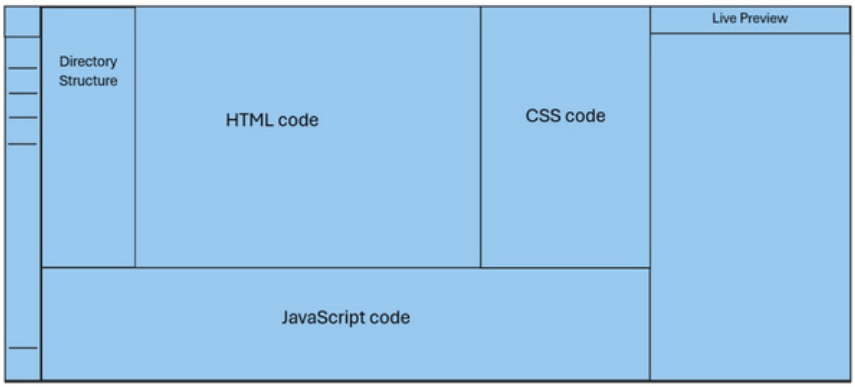
- Enables multi-language learning and testing in one environment.
- Built-in Compilation, no extra installation.

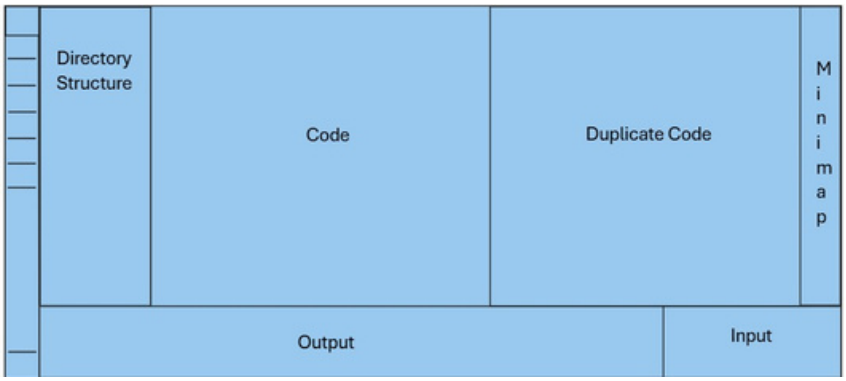
Desktop app UI in progress

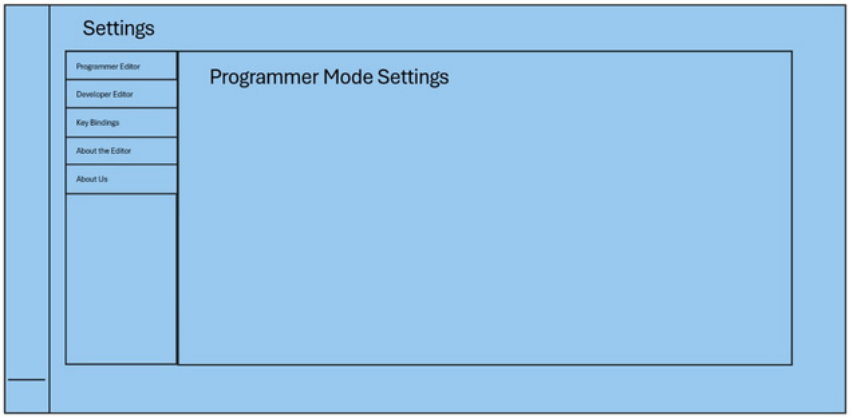
Desktop app UI of Soniverse is in progress, the blueprint of the app are the below images there will be 4 parts in app -

- Welcome page,
- Programmer mode,
- Developer mode
- Settings panel

Welcome page is displayed at 1st, once u enter the editor **Programmer mode** is displayed, we can switch between it and **Developer mode**, and there will be **Settings panel** to customize themes, keybindings, programmer editor, developer editor and information about the editor and its license







15. HTML to JavaScript (DOM Conversion)

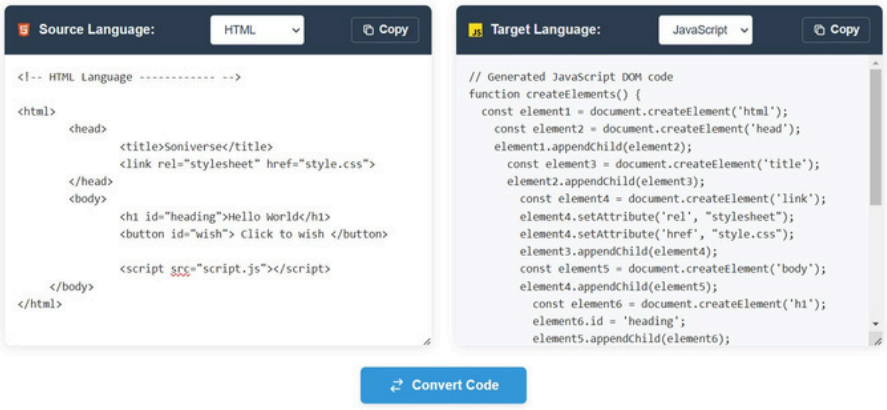
HTML to JavaScript conversion involves turning HTML structure into JavaScript code using DOM methods like createElement, appendChild, and setAttribute. This helps dynamically generate HTML content via JavaScrip

Features:

- Converts tags like <h1>, <button>, etc., into JavaScript.
- Uses functions like document.createElement().
- Adds attributes and appends elements via JS.
- Good for creating elements on the fly.

Advantages:

- Useful in single-page applications (SPA).
- Helps generate dynamic content without reloading the page.
- Great for frameworks like React or vanilla JS UIs.
- Promotes separation of logic and structure.



How it works

1. Input Processing: Reads the HTML line-by-line and trims unnecessary whitespace.

2. Element Creation:

- Detects each opening tag and uses document.createElement() to create elements in JavaScript.
- Assigns id and class using .id and .className.
- Uses .setAttribute() for other attributes.

3. Text Content Handling: Detects any plain text and assigns it to .textContent of the element.

4. DOM Nesting:

- Tracks nesting using indentation logic.
- Appends child elements using .appendChild() to mimic HTML structure.

5. Final Output:

- Wraps all logic inside a function called createElements().
- Automatically appends the structure to document.body.

16. HTML to CSS (Skeleton Conversion)

HTML to CSS conversion auto-generates a CSS "skeleton" from HTML, creating selectors for each element or tag found in the HTML file. You can then style them as needed.

Features:

- Detects HTML tags and IDs/classes.
- Creates CSS blocks for each.
- Organizes layout-ready styles.
- Helps beginners structure styling.

Advantages:

- Saves time in writing selectors.
- Encourages clean and modular CSS.
- Acts as a blueprint for styling.
- Especially useful for beginners to learn selector mapping..



How HTML to CSS Converter Works:

1.Input Parsing: Takes the HTML code and scans it for all tags, classes, and IDs.

2. Element Extraction:

- Uses regex to identify all HTML tags (like , <p>).
- Finds all classes defined using class="...".
- Finds all IDs using id="...".

3. Output: Returns the generated CSS structure as a readable template.CSS

4. Skeleton Generation:

- Creates empty CSS blocks for each tag, class, and ID.
- For example, results in:

```
div
 {
   /* Element styles */
 }
.box
 {
   /* Class styles */
 }
#main
 {
   /* ID styles */
 }
```

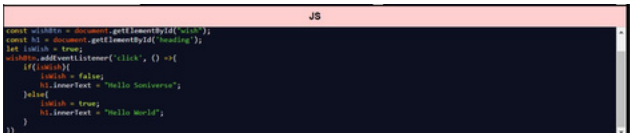
17. HTML, CSS, JS Code Editors

The HTML, CSS, and JavaScript code editors are integrated tools designed for front-end web development. Each editor focuses on a specific language: HTML structures the content, CSS styles it, and JavaScript adds dynamic behavior. These editors provide a clear and organized layout, making it easy to write, test, and debug code. Syntax highlighting and formatting features help developers avoid mistakes and understand code quickly. This setup is ideal for learning and fast web prototyping.

Features:

- Syntax highlighting for each language (HTML, CSS, JS).
- Separate panels for clean and organized coding.
- Real-time code updates across all editors.
- Easy integration with external files like stylesheets and scripts.
- Simple interface with built-in file management options.





Advantages:

- Encourages clean code structure and modular design.
- Helps beginners learn by separating structure, style, and behavior.
- Increases productivity with side-by-side code editing.
- Reduces dependency on external IDEs.
- Speeds up the development and testing process.

18. Live Preview Panel

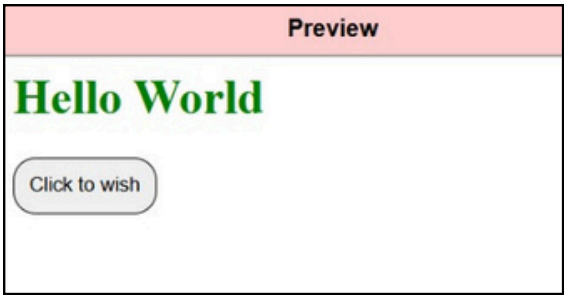
The live preview panel is an embedded area that shows the output of your HTML, CSS, and JavaScript code in real time. It allows you to see exactly how your webpage will appear in a browser. As soon as you make a change in the code, the results are instantly visible. This helps you understand the immediate effect of your edits. It's a powerful tool for learning, testing, and debugging web designs quickly.

Features:

- Instant display of code output without reloading.
- Real-time interaction with JavaScript features.
- Reflects CSS styling changes immediately.
- Mimics the look and feel of a web browser.
- Works in sync with the code editor panels.

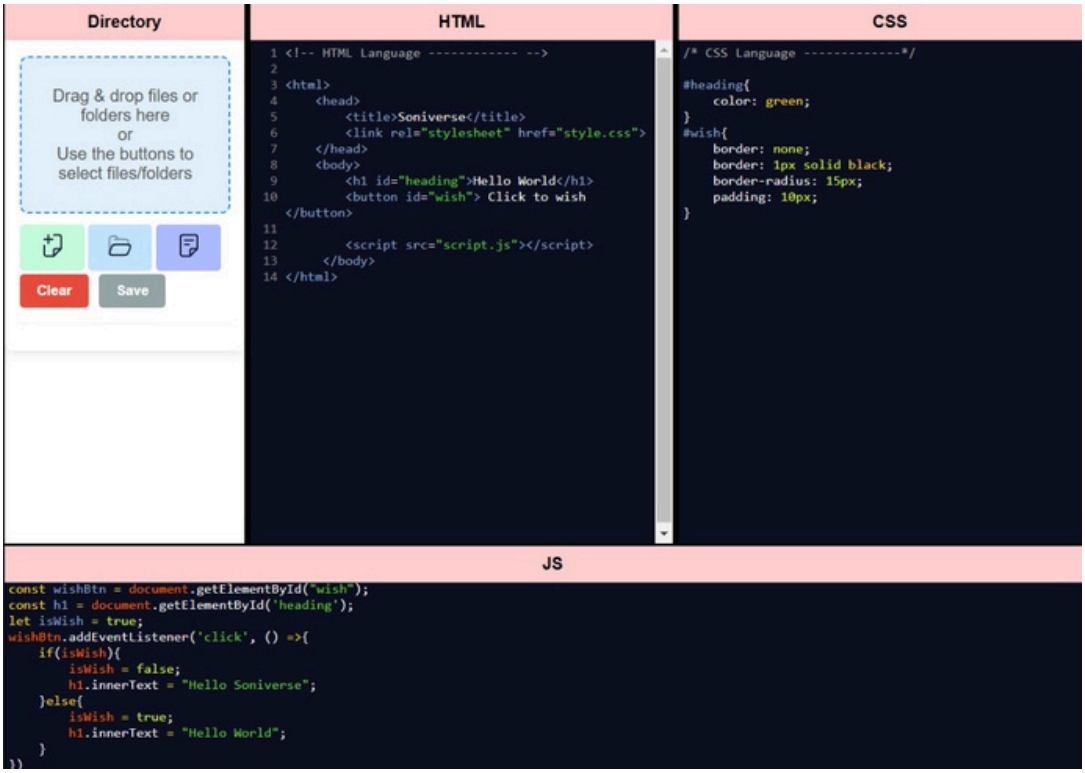
Advantages:

- Saves time by removing the need to switch to an external browser.
- Makes learning easier through immediate visual feedback.
- Helps in quick debugging and layout testing.
- Encourages experimentation with code changes.
- Ideal for rapid prototyping and design validation.



19. Seperate directory structure :

it includes - select folder, select multiple files, create new files
Only difference is, files with .html, .css, .js extensions will emit code on html editor, css editor, js editor respectively.



Advantages:

- **Organized Structure –** Keeps files grouped (HTML, CSS, JS) for better project management.
- **Auto Editor Routing –** Opens .html, .css, .js files in respective editors automatically.
- **Scalable –** Suitable for larger, multi-file projects.
- **Efficient Workflow –** Supports folder/multi-file selection and quick file creation.
- **Boosts Productivity –** Easy switching and editing across related files.
- **Best Practices –** Encourages separation of concerns (structure, style, logic).
- **IDE-like Feel –** Mimics modern development environments in the browser.

20.AI Chatbot

Introduction:

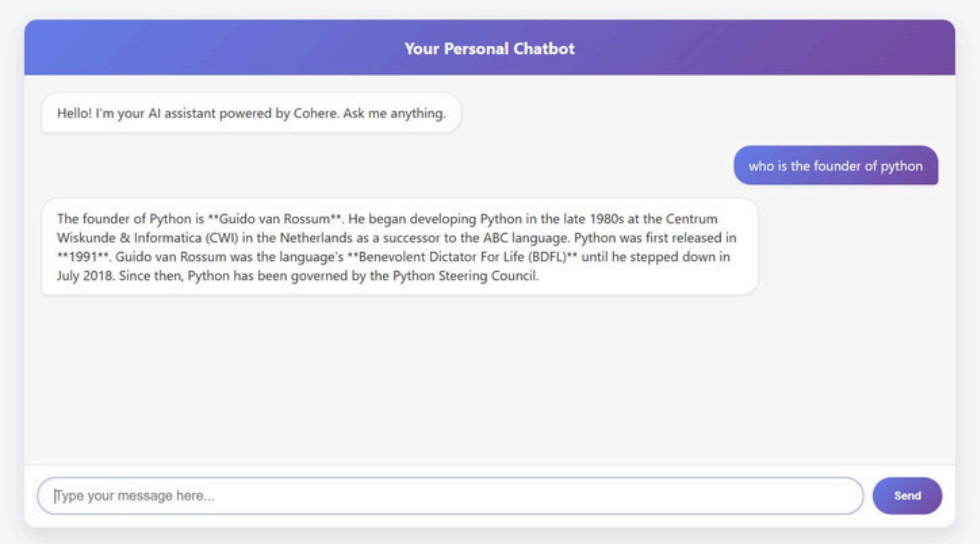
An AI chatbot is a computer program that uses artificial intelligence (especially Natural Language Processing or NLP) to simulate human conversation. It can understand text or voice inputs and respond intelligently, just like a human

Features:

- Understands user queries in natural language.
- Responds smartly using AI algorithms.
- Can work 24/7 without any break
- Easily integrated into websites, apps, or messaging platforms.

Advantages:

- Saves time by answering questions instantly.
- Reduces workload on human support teams.
- Improves customer service and user experince.
- Learns and improves over time using machine learning.



How It Works:

This AI Chat Assistant uses Hugging Face models to respond to user messages. Here's how it works step by step:

1. Getting HTML Elements

- The script connects to important parts of the web page like:
- Chat area (chatMessagesAI)
- Message box (messageInputAI)
- Send button (sendButtonAI)
- API key input (apiKeyInputAI)
- Model selector (modelSelectAI)

2. Showing Messages:

- When a message is sent or received:
- A new message box is created.
- It's styled based on who sent it (user or AI).
- It is added to the chat area.
- The chat scrolls to the latest message.

3. Typing Dots :

- When the AI is thinking:
- A typing animation (three dots) is shown.
- It disappears once the AI replies.

4. Talking to the AI :

- The main function (sendToAIAssistant) does the following:
- Checks if the API key is entered.
- Prepares the message depending on the model chosen (e.g., Mistral, Bloom, Flan-T5).
- Sends the message to the Hugging Face API.
- Receives and shows the AI's reply.

5. User Inputs :

- The assistant listens for:
- Clicking the send button.
- Pressing Enter on the keyboard.
- Once the user types something:
- It is shown in the chat.
- The AI is asked to respond.

6. Handling Errors :

- If something goes wrong (like model loading or API issues):
- An error message is shown in the chat.
- The user is advised to try again later.
- The script connects to important parts of the web page like:
- Chat area (chatMessagesAI)
- Message box (messageInputAI)
- Send button (sendButtonAI)
- API key input (apiKeyInputAI)
- Model selector (modelSelectAI)

21.Electron App Overview

Introduction:

Electron is an open-source framework that allows developers to build cross-platform desktop applications using web technologies like HTML, CSS, and JavaScript. For this project, Electron wraps the collaborative code editor into a standalone desktop application, providing a native-like experience without the need for a browser.

Features:

- Operating System Compatibility
- Electron apps are compatible with all major desktop platforms:
- Windows (7 and above, 64-bit recommended)
- macOS (10.12 Sierra and later)
- Linux (Ubuntu, Debian, Fedora, Arch, etc.)
- Electron packages each app with its own Chromium engine and Node.js runtime, ensuring consistent performance across operating systems.

Advantages:

- **Cross-Platform Support:** Develop once and run anywhere – Windows, macOS, and Linux.
- **Offline Capability:** The app can function offline with local file storage and session caching.
- **Native-Like Experience:** Offers better performance and tighter OS integration than web apps.
- **Auto Updates & Installation:** Easy to distribute via installers; supports auto-updating.
- **Access to System Resources:** Enables access to native system APIs (e.g., file system, notifications)

Welcome to Soniverse

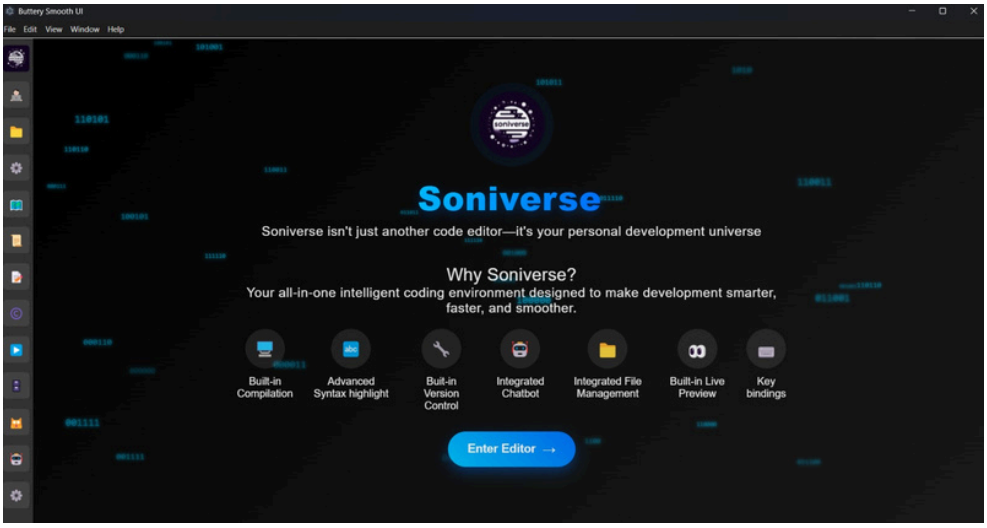
Soniverse is more than just a code editor. It's a complete, intelligent environment built to make coding simple, smooth, and efficient. Whether you're learning to code or building complex applications, Soniverse offers tools that support every step of the journey – online or offline.

i. Home Page

The first interaction users have with Soniverse is its intuitive home page. It lets you easily choose between Programmer or Developer mode.

Advantages:

- Clean UI for easy navigation
- Quick mode switching
- Direct access to your favorite coding tools

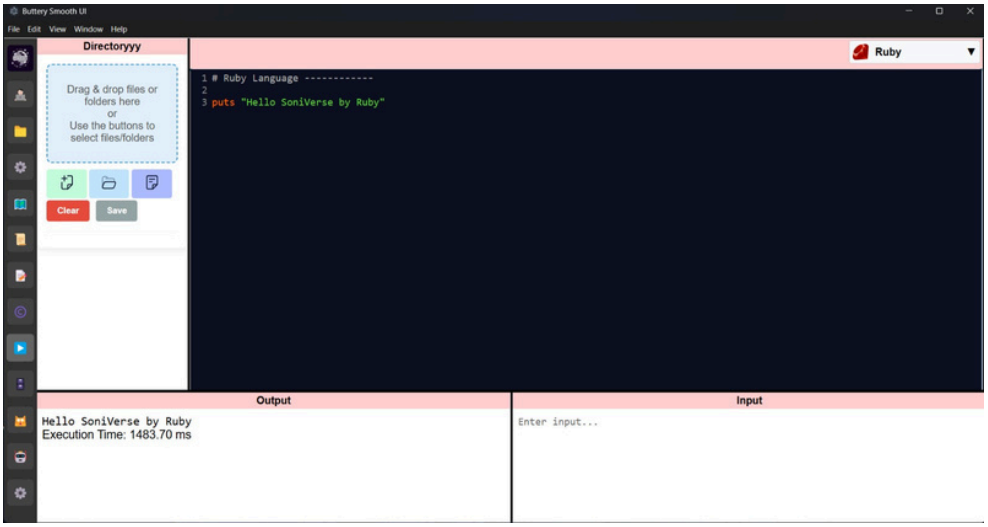


ii. Programmer Mode

This mode is shown in the image above. It includes a language selector (like Ruby), file handling options, a code editor, and real-time output display.

Features:

- Editor
- Duplicate pannel
- Input section
- Output section
- Directory structure
- Language dropdown
- Switch to developer mode
- version control
- Ai chatbot

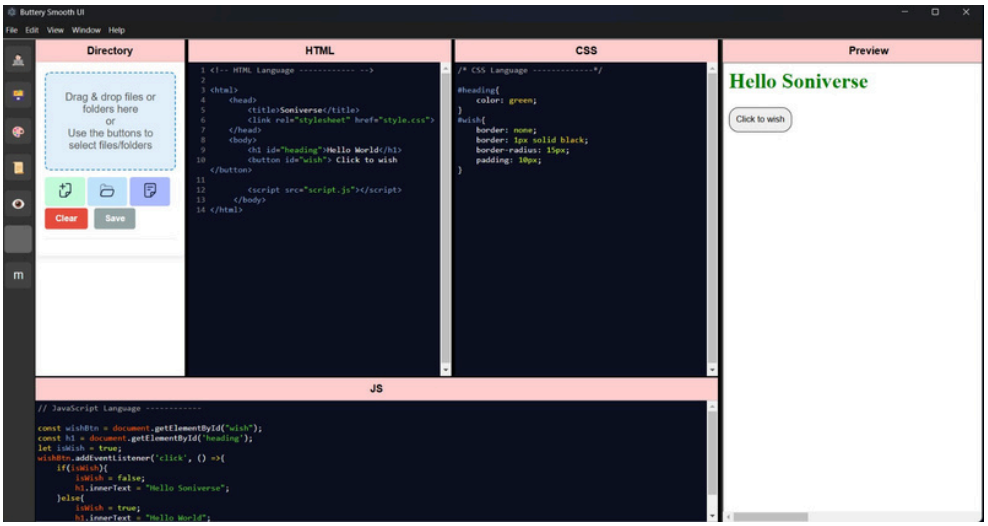


iii. Developer Mode– Build, Style, and Execute Like a Pro

Developer Mode in Soniverse is designed for users who want a more structured, hands-on development experience. It gives you full control of the front-end stack (HTML, CSS, JS) with real-time results and flexible code transformation tools. Perfect for front-end devs and UI designers.

Separate Directory Structure:

- html css js editor
- live preview
- code converter



iv. Settings Panel – Key Feature Introduction

-The Settings Panel acts as the central control hub for customizing the code editor experience.

-It includes access to both the Programmer Editor and Developer Editor, each tailored for different coding needs.

-This panel allows seamless switching between coding styles and user preferences.

-With the Customize Key Bindings option, developers can personalize keyboard shortcuts for faster workflows.

-It enables intuitive control over editor behavior, enhancing coding comfort and efficiency.

-Users can learn more through the About the Code Editor section, offering insight into editor functionality.

-The About Us section helps users understand the creators and purpose of the platform.
-Its organized layout ensures quick navigation to any tool or setting.
-Whether you're a front-end developer or backend programmer, this panel adapts to your workflow.
-Overall, the Settings Panel makes the editor powerful, flexible, and user-friendly for all developers.

a. Programmer mode editor Settings

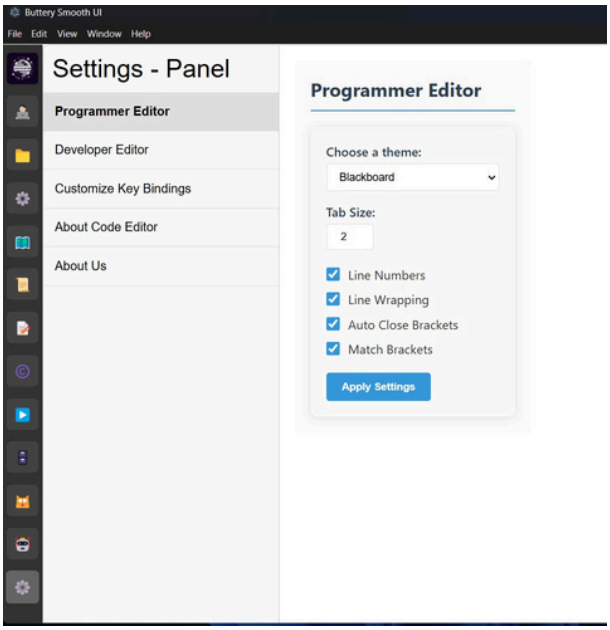
The Programmer Editor is a simplified yet powerful code editor setup for general-purpose or backend coding. It includes basic customization settings such as theme selection, tab size, and essential editing features like line numbers and bracket matching. This editor is ideal for those who focus on scripting, logic building, or back-end development.

Features:

- Theme selection to choose a coding-friendly interface (e.g., Blackboard).
- Tab size customization (e.g., 2 spaces for compact coding style).
- Line numbers display toggle for easier debugging and navigation.
- Auto-close brackets to reduce typing errors.
- Bracket matching for better understanding of code structure.

Advantages:

- Ideal for programmers focusing on logic-heavy tasks or scripts.
- Provides a clean interface with only essential features.
- Reduces development time with auto-close and match functions.
- Supports precise indentation and consistent formatting.
- Increases readability and reduces logical errors in complex code



b. Developer mode editor settings

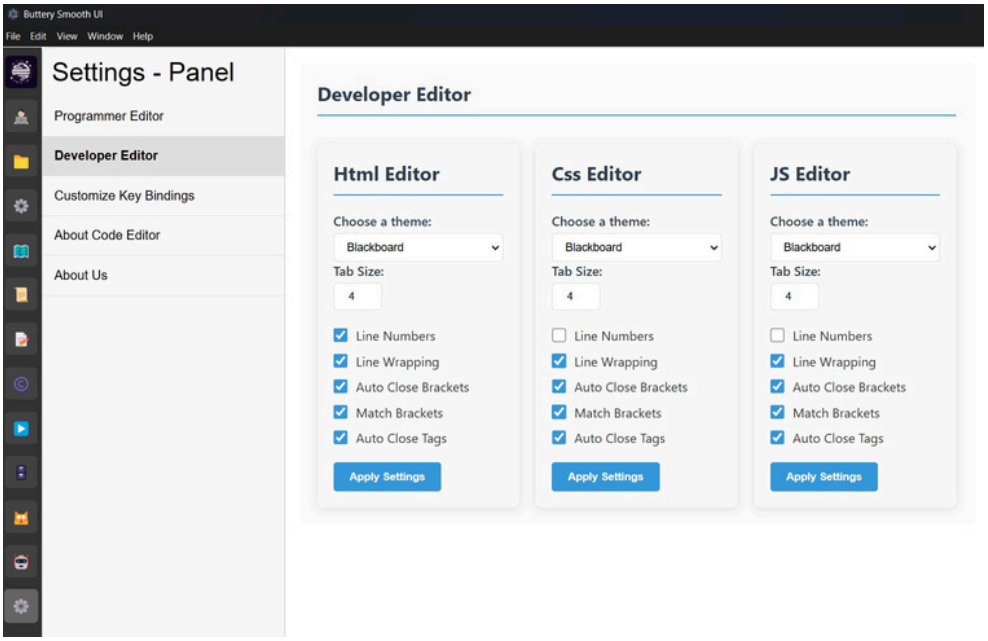
The Developer Editor is a customizable web-based code editor interface specifically designed for front-end development. It includes individual settings panels for HTML, CSS, and JavaScript editors, enabling developers to adjust their coding environment to their liking. Each editor allows changes to the theme, tab size, and coding assistance options like line numbers, auto close brackets, and match brackets or tags.

Features:

- Theme selection for HTML, CSS, and JS editors (e.g., Blackboard).
- Adjustable tab size (e.g., 4 spaces for consistent formatting).
- Toggle line numbers to show/hide line count for better code navigation.
- Auto-close brackets and tags to speed up writing HTML and JS.
- Match brackets visually highlights paired brackets in code.

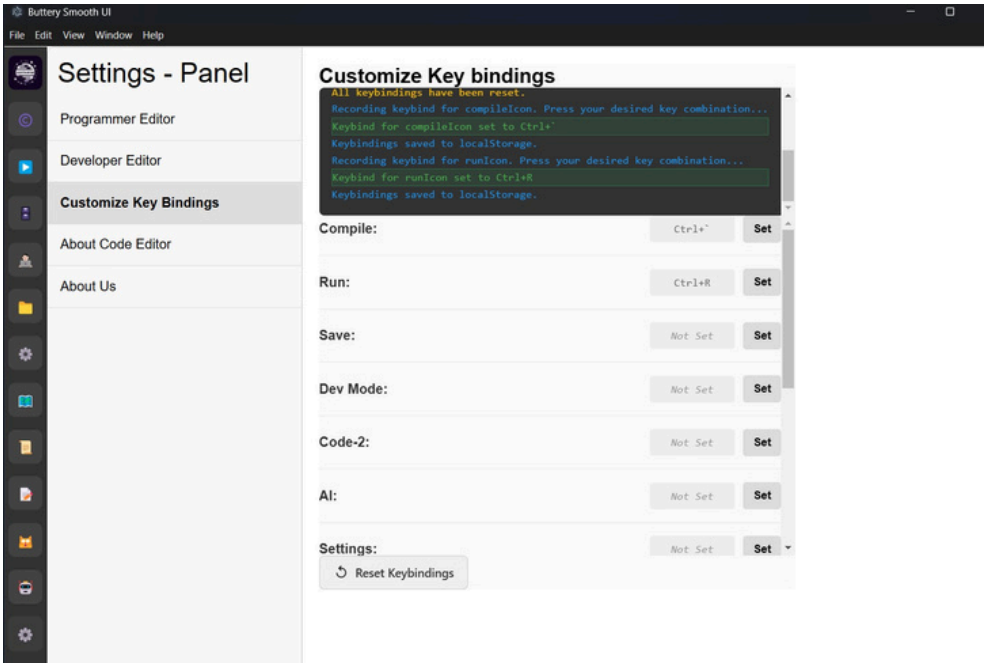
Advantages:

- Improves workflow with customized settings for each language.
- Enhances accuracy with features like auto-close and bracket match.
- Reduces visual clutter by toggling unnecessary tools.
- Boosts learning by helping beginners understand syntax structure.
- Encourages efficient, error-free coding across all three front-end languages.



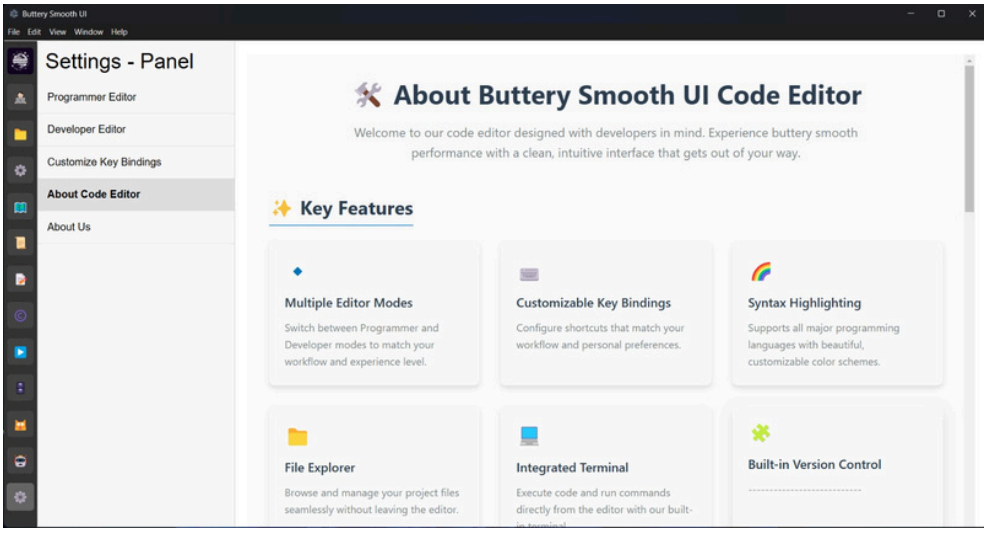
c. The Customize Key Bindings

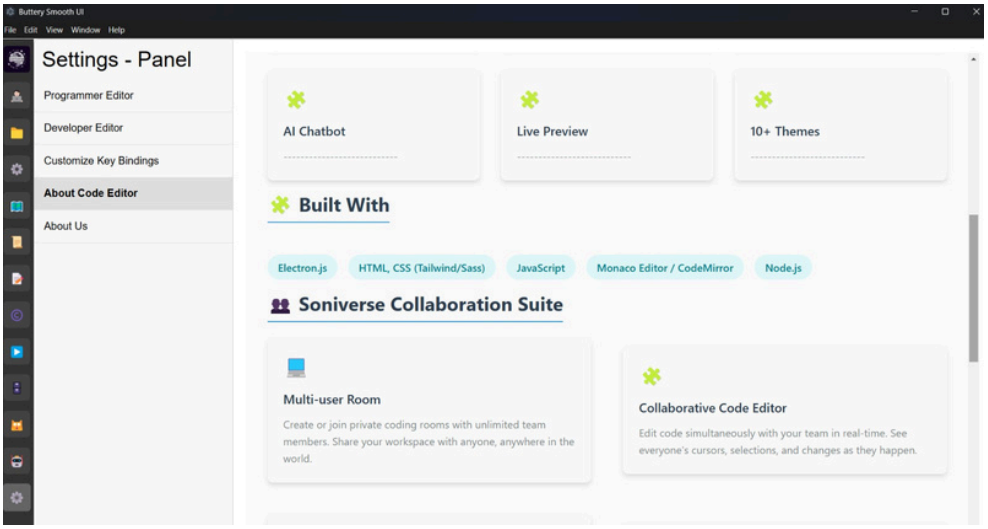
The Customize Key Bindings feature allows users to assign or change keyboard shortcuts for various actions within a software application. This enhances user experience by offering greater control, efficiency, and personalization, especially in coding or collaborative environments.

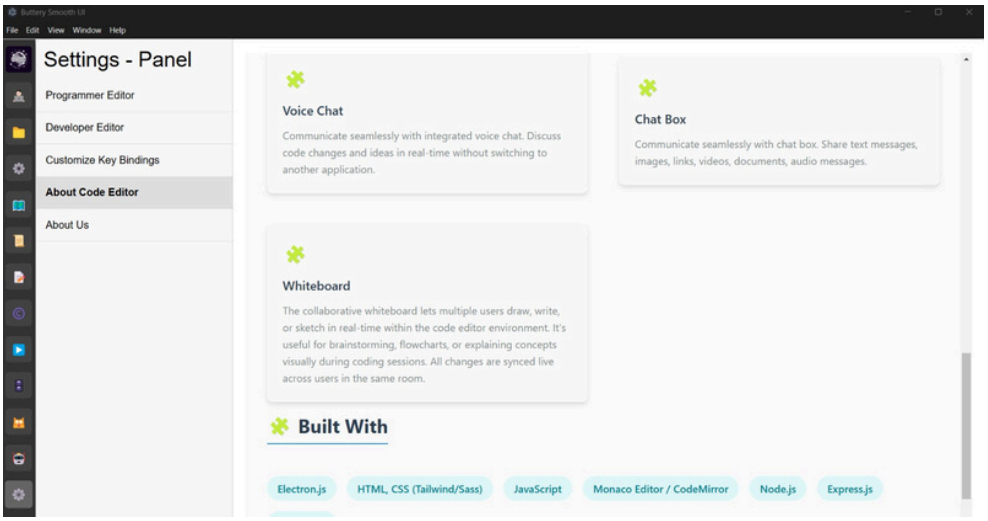


d. About Code editor

It includes key features of the Electron application and key features of Web application with real time features.

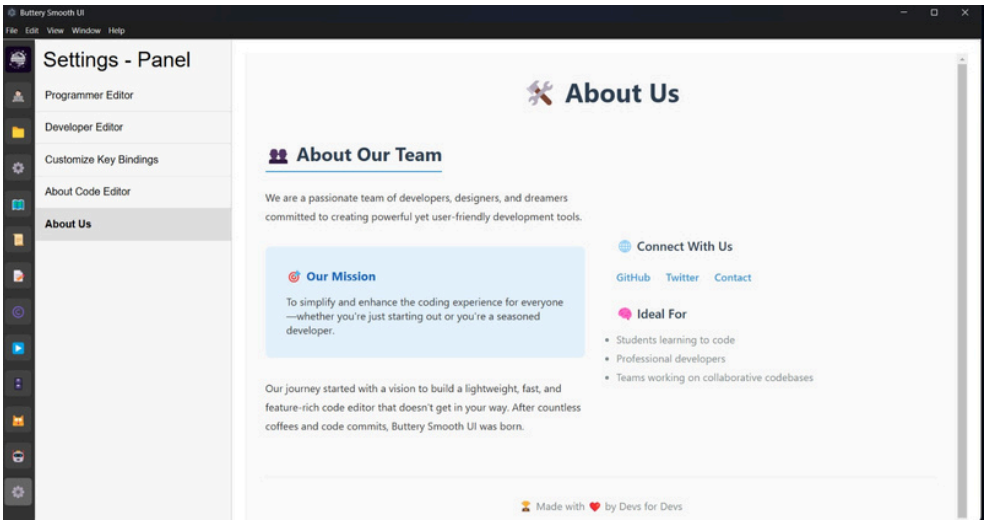






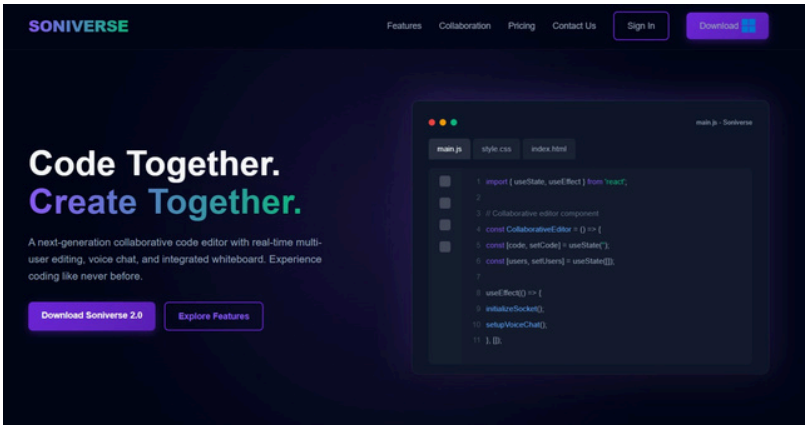
e. About Us :

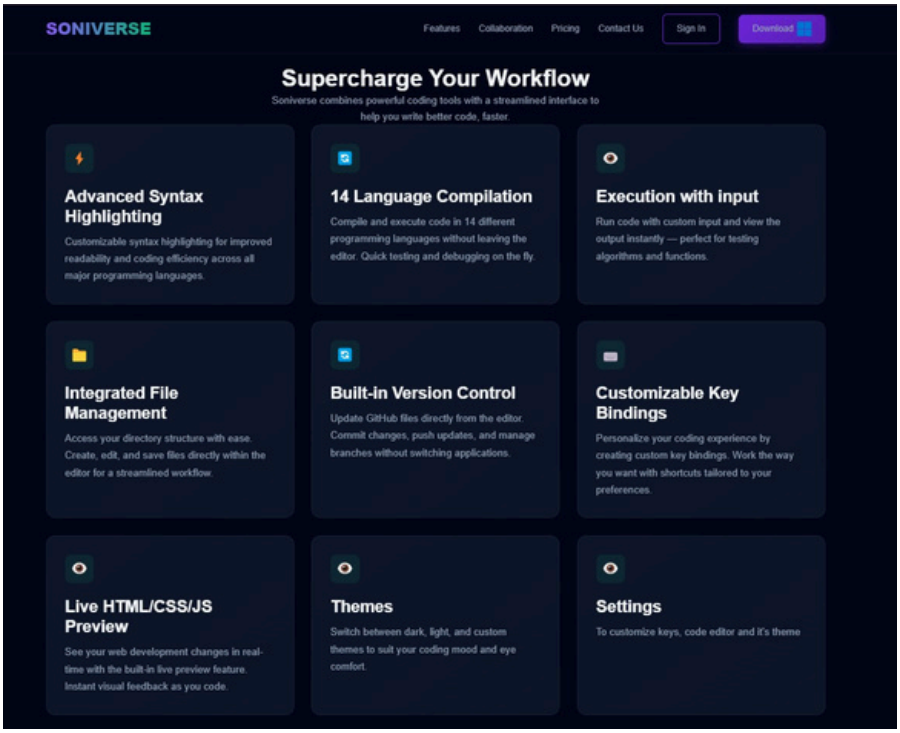
This section includes Developers intentions and inspiration to develop the code editor.

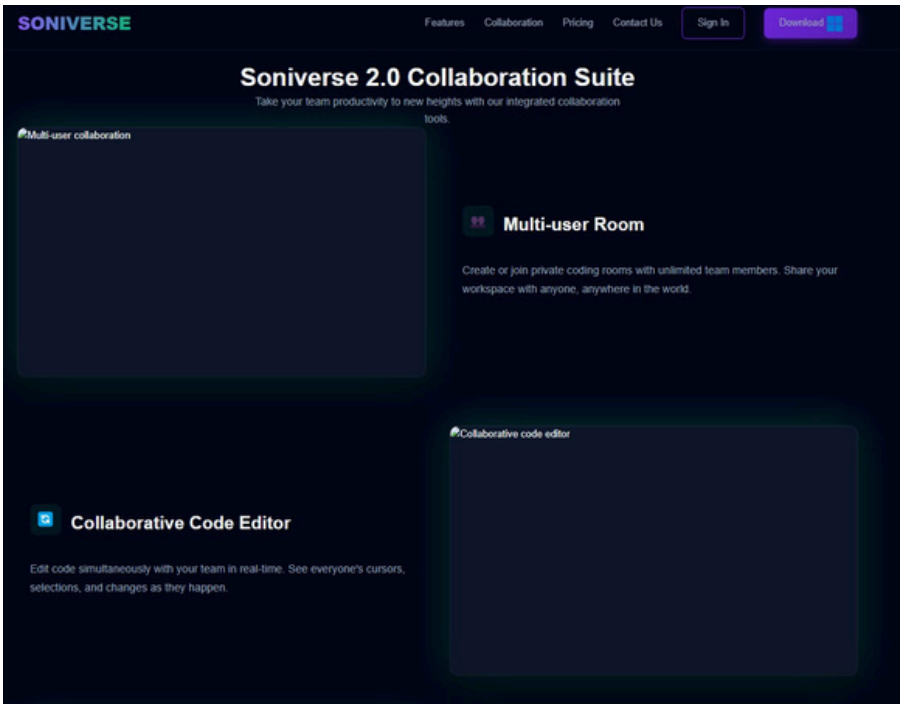


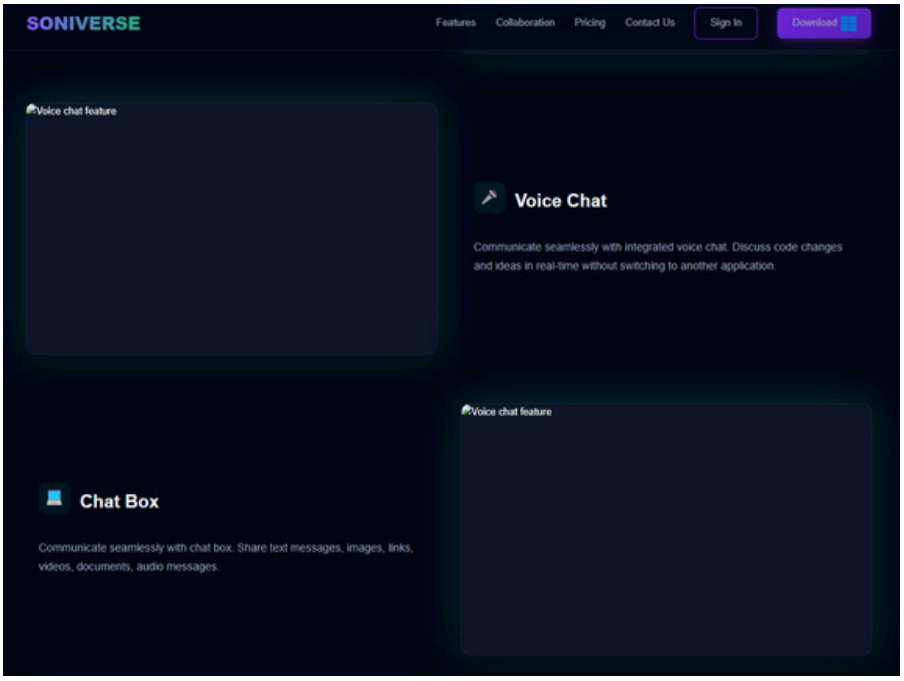
22. Soniverse Website

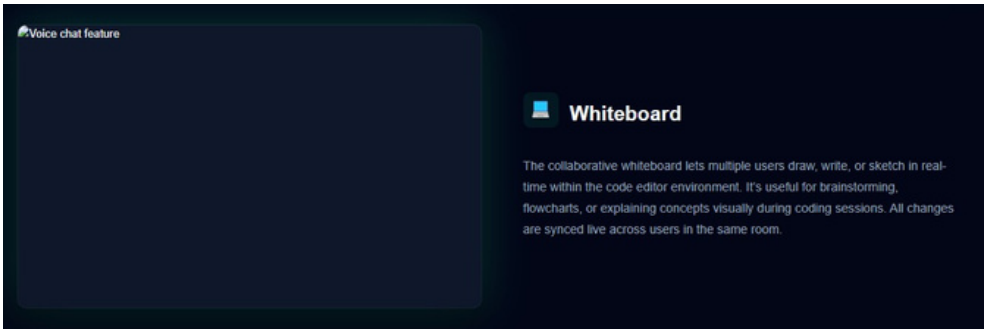
Welcome to Soniverse –
Soniverse is a next-generation collaborative code editor that transforms the way teams build software.
Experience real-time multi-user editing, seamless voice chat, and an integrated whiteboard for brainstorming and planning–all in one powerful platform.
Whether you're working remotely or side-by-side, Soniverse keeps everyone in sync and coding with clarity.
Enjoy a sleek interface with support for JavaScript, CSS, and HTML, designed for modern developers. Unlock a new level of productivity and creativity–download **Soniverse 2.0** and start building together.





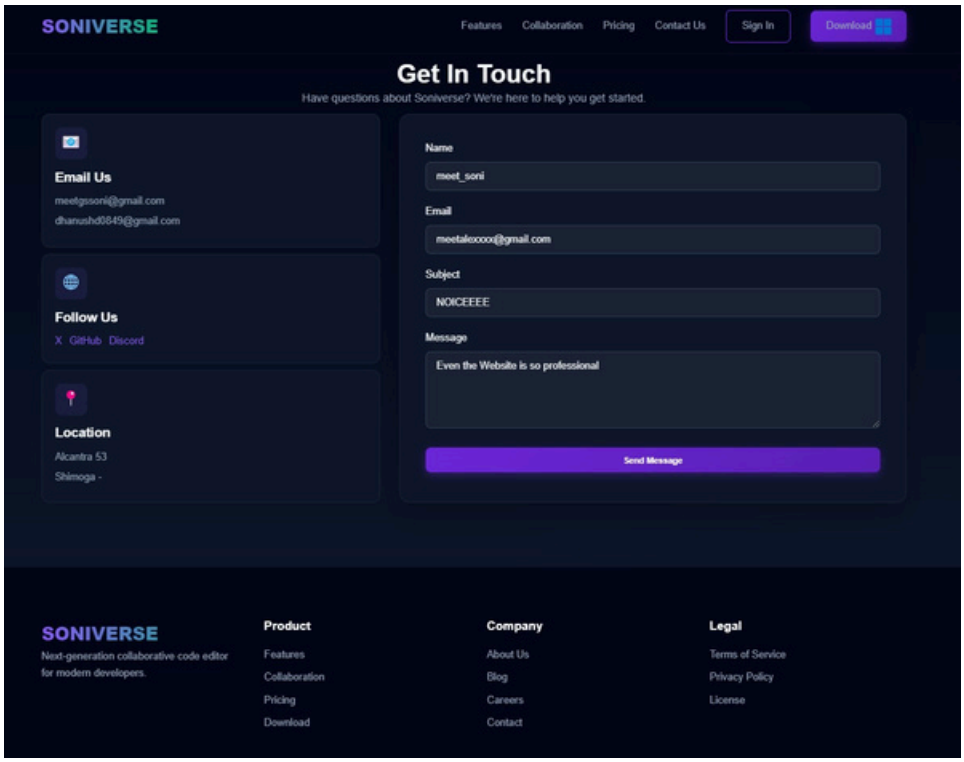

SONIVERSE
Features
Collaboration
Pricing
Contact Us
Sign In
Download
Soniverse 2.0 Collaboration Suite
Take your team productivity to new heights with our integrated collaboration tools.
Multi-user collaboration
Multi-user Room
Create or join private coding rooms with unlimited team members. Share your workspace with anyone, anywhere in the world.
Collaborative code editor
Collaborative Code Editor
Edit code simultaneously with your team in real-time. See everyone's cursors, selections, and changes as they happen.

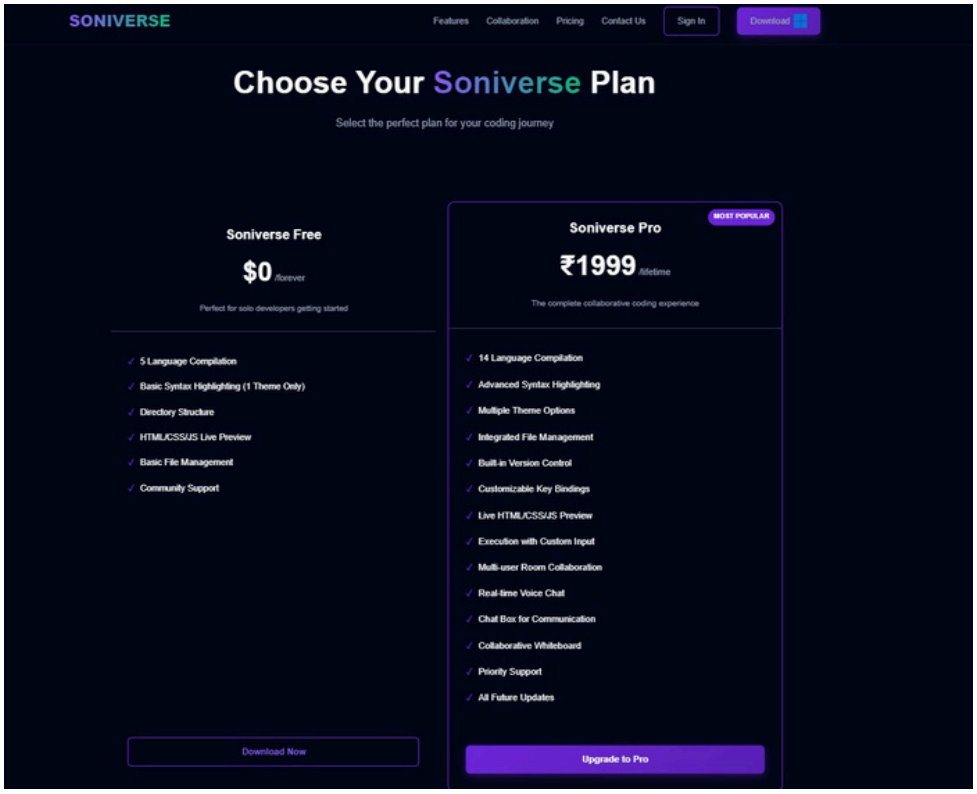

SONIVERSE
Features
Collaboration
Pricing
Contact Us
Sign In
Download
Voice chat feature
Voice Chat
Communicate seamlessly with integrated voice chat. Discuss code changes and ideas in real-time without switching to another application.
Voice chat feature
Chat Box
Communicate seamlessly with chat box. Share text messages, images, links, videos, documents, audio messages.


Voice chat feature
Whiteboard
The collaborative whiteboard lets multiple users draw, write, or sketch in real-time within the code editor environment. It's useful for brainstorming, flowcharts, or explaining concepts visually during coding sessions. All changes are synced live across users in the same room.

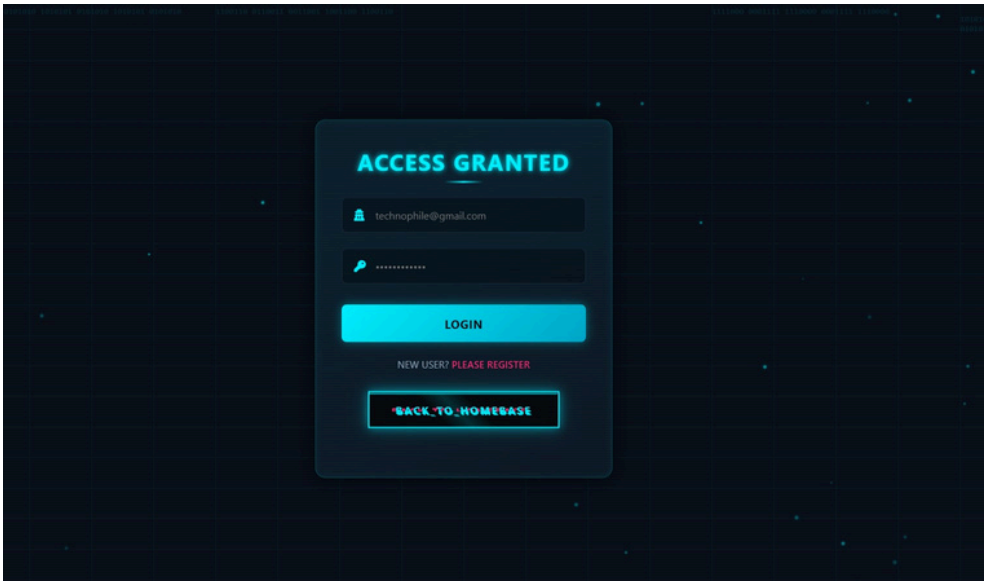
Below is a get in touch page in case any user wants to contact the developer of app and website

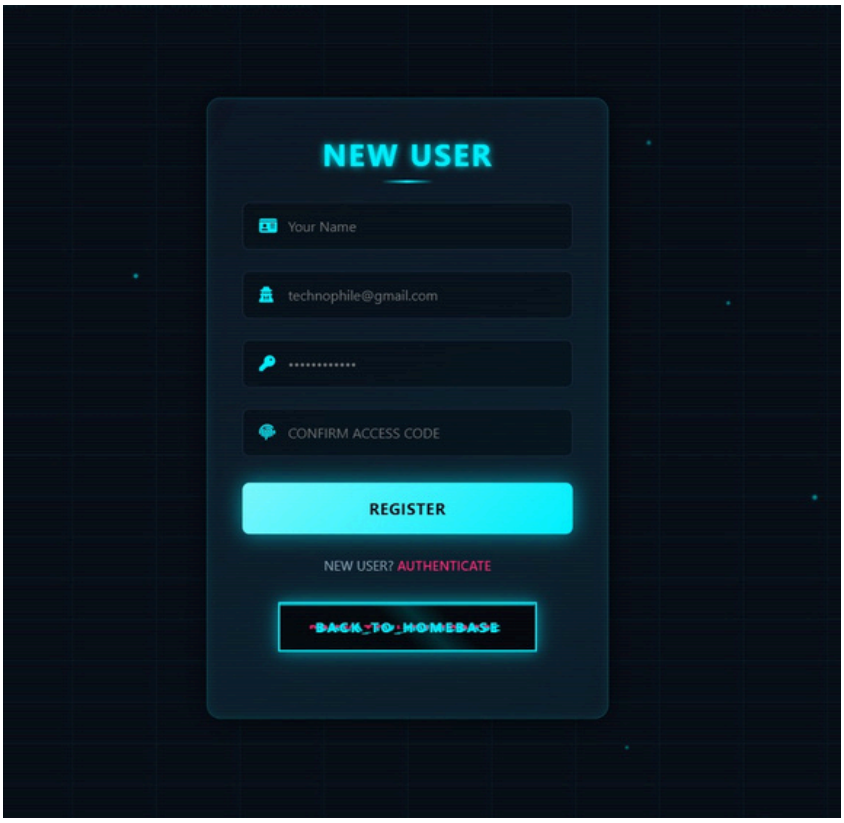
Pricing page shows 2 plans, 1st one is a free app, 2nd one is a paid app with mentioned features





Also a Login and Signup page been desgined, in order to get any website visiter data with name, email, password.

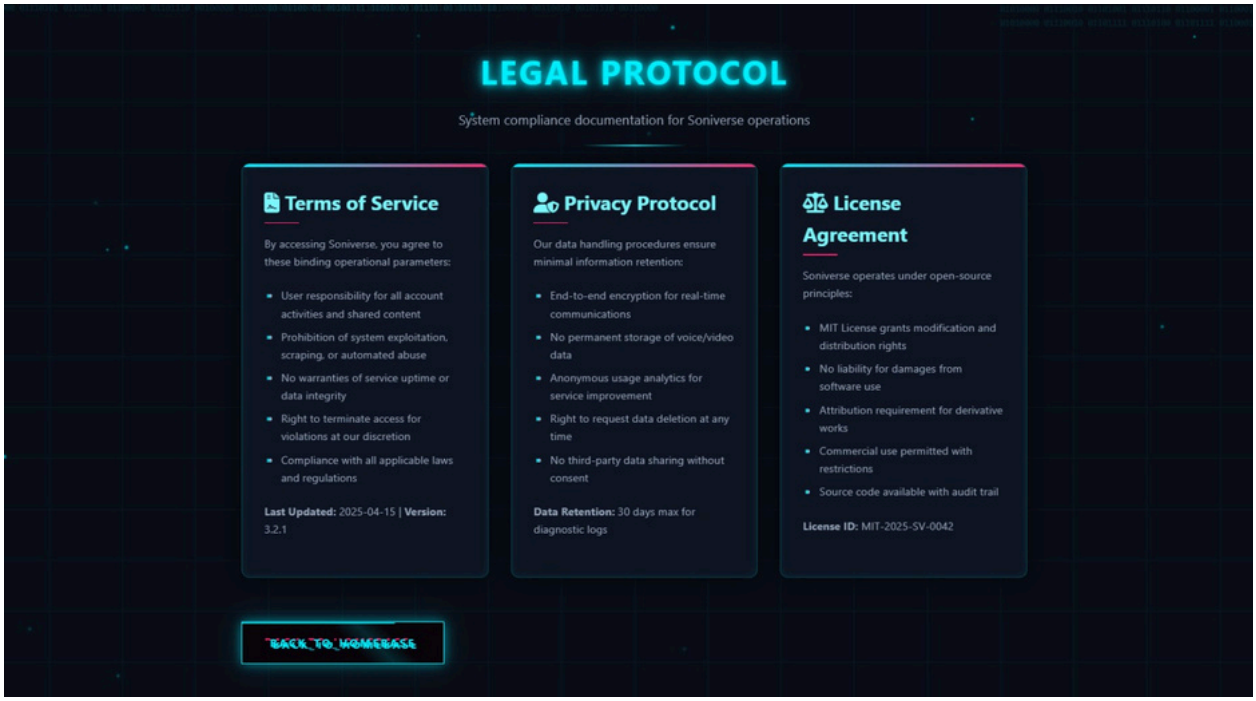




The Legal Protocol :

The Legal Protocol page outlines the compliance framework for Soniverse operations, detailing the terms of service, privacy protocols, and licensing agreements. This page is essential for ensuring that users understand their rights and responsibilities while using the platform, as well as the operational guidelines that govern the service.

- Terms of Service: - Clearly states user responsibilities for account activities. Prohibits unauthorized actions like system exploitation or automated abuse. Details the right to terminate accounts for violations.
- Privacy Protocol - Describes data handling procedures to protect user information. Emphasizes end-to-end encryption for real-time communications. Outlines user rights regarding data usage and deletion. Specifies data retention policies for diagnostics.Mimics the look and feel of a web browser.
- License Agreement - Clarifies that Soniverse operates under open-source principles. Details the MIT License provisions for modification and distribution. States liability disclaimers for software use.Mentions requirements for attribution in derivative works.



1. **html css js editors**
2. **live preview**
3. **html to css**
4. **html to Js**
5. **ai chatbot**
6. **electron app details**
7. **home page of electron app**
8. **Programmer mode**
9. **developer mode**
10. **settings panel - 3 line intro**
11. **programmer mode settings**
12. **dev mode settings**
13. **about the code editor**
14. **about Us**
15. **soniverse full website**